

SAI 1 — SYSTEM ARCHITECTURE AND INTEGRATION 1 (ITNET312-T)

COMPLETE EXAM REVIEWER — LESSONS L2 to L8

Source: SAI Review Reference · Prof. Pops Madriaga · 3rd Year 1st Sem · BSIT-S-3A

Lessons covered: **L2** – Enterprise Architecture Components · **L3** – Developing an Enterprise Architecture · **L4** – Using an Enterprise Architecture · **L5** – All About SOA · **L6** – Integrating Technology · **L7** – Integrating Applications · **L8** – Enterprise Integration

L2: ENTERPRISE ARCHITECTURE COMPONENTS

MUST MEMORIZE — Key Terms & Definitions

EA Implementation Methodology — a detailed, step-by-step approach that describes how an Enterprise Architecture program is established, developed, managed, maintained, and used within an organization. It provides a clear approach for building and managing an effective EA program.

Importance of a Structured Implementation Methodology — helps an organization establish and manage its EA program in a clear, consistent, and coordinated way.

Key Benefits (6)

1. **Clear Direction** — defines the EA's scope, goals, and overall approach.
2. **Better Governance** — establishes how EA decisions are managed and coordinated.
3. **Organized Documentation** — identifies what EA information and artifacts need to be documented.
4. **Stakeholder Alignment** — helps stakeholders understand the EA's purpose, value, and direction.
5. **Better Decision-Making** — provides organized EA information that supports planning and decisions.
6. **Continuous Improvement** — supports regular updates and maintenance as the organization changes.

EA Framework — a structured set of tools, models, and guidelines used to organize and design an enterprise's architecture. It provides a common language and structure for planning, building, and managing IT and business systems, and helps ensure consistency across all levels of the organization.

Why organizations need a framework

- Reduces complexity and miscommunication
- Aligns business goals with IT strategy
- Improves decision-making and resource planning
- Provides consistency, especially in large or growing organizations

The 4 Phases of EA Implementation (LIST TO MEMORIZE)

1. **PHASE I — EA Program Establishment** — establish the EA program and leadership; define the implementation methodology; establish governance; communicate the EA vision and gain stakeholder support.
2. **PHASE II — Framework and Tool Selection** — select the EA documentation framework; identify Lines of Business and Crosscuts; identify EA components to document; select documentation methods and tools; establish an EA repository.
3. **PHASE III — Documentation of the EA** — evaluate existing documentation; document the current state; develop future operating scenarios; define future planning assumptions; document the future state; develop the EA Management Plan.
4. **PHASE IV — Use and Maintain the EA** — use EA documentation for planning and decision-making; regularly update current and future views; maintain the EA repository and tools; update the EA Management Plan.

Application of EA Implementation Methodology (Example)

Situation: a planning team needs to identify improvements in information exchange within a Line of Business.

Using the EA Repository: access current and future views of the information flow; review related business services, applications, and networks; display the EA diagrams and information during the meeting.

Result: faster communication of ideas, better understanding among stakeholders, fewer interpretation errors, more productive discussions, better-informed decisions.

Benefits and Challenges (MEMORIZE)

Benefits: Clearer Direction, Better Governance, Improved Understanding, Better Communication, More Coordinated Planning, Continuous Improvement.

Challenges: Complexity, Governance Difficulties, Requirements Gaps, Implementation Problems, Maintenance Issues.

METHODOLOGY vs FRAMEWORK (COMPARISON)

Aspect	EA Implementation Methodology	EA Framework
Question it answers	HOW the EA program is established and managed	WHAT STRUCTURE is used to organize the EA
Focus	Defines processes, steps, and activities	Organizes architecture information
Role	Guides development, documentation, and maintenance	Defines the scope and relationships of architecture areas
Includes	Framework and tool selection	Provides the structure used for documenting the EA

ZACHMAN FRAMEWORK

Zachman Framework — developed by **John Zachman in 1987**; one of the earliest and most influential EA frameworks. It is **not a methodology** — it is a **classification schema** (a way of organizing and categorizing architectural artifacts). Often compared to **blueprints** used in other engineering disciplines.

Structure (6x6 matrix, 36 cells total)

- **6 Columns (Focus / Interrogatives):** What, How, Where, Who, When, Why
- **6 Rows (Perspectives):** Planner, Owner, Designer, Builder, Sub-Contractor, Enterprise (Actual System)
- Each cell represents a specific artifact or model relevant to that intersection.

How it is used / Key takeaways

- Helps organizations ensure completeness — no perspective or aspect is overlooked.
- Used as a **reference/checklist, not a step-by-step process**.
- Encourages clear communication between different stakeholders (executives, designers, technical teams).
- Still influences modern frameworks like TOGAF today.

TOGAF (THE OPEN GROUP ARCHITECTURE FRAMEWORK)

TOGAF — an Enterprise Architecture framework used to help organizations **design, plan, implement, and manage** their enterprise architecture. It provides a structured approach for developing architectures and is widely used to align business goals with IT systems and technology.

Purpose of TOGAF (LIST)

- Align IT with business objectives
- Plan changes to systems and technology
- Identify gaps between the current and desired architecture
- Reduce duplication and unnecessary technology
- Improve communication between business and IT teams
- Manage architecture changes in a structured way

Example: a company wants to move from manual processes to an online system — TOGAF helps analyze the current situation, design the target system, and plan how to transition.

ADM — Architecture Development Method

The ADM helps organizations (6 steps):

1. Identify business requirements
2. Analyze the current architecture
3. Design the target architecture
4. Identify the changes needed
5. Plan implementation
6. Manage and review the architecture

TOGAF ADM CYCLE (MEMORIZE)

Phase	Key Focus
Preliminary Phase — Preparation	Establish EA principles and governance; define roles, responsibilities, and processes
Phase A — Architecture Vision	Define goals and scope; identify stakeholders; create the overall architecture vision
Phase B — Business Architecture	Define business goals, processes, and organization
Phase C — Information Systems Architecture	Data: identify and organize required data; Applications: define systems and how they interact
Phase D — Technology Architecture	Define infrastructure, networks, platforms, and technologies
Phase E — Opportunities & Solutions	Identify possible solutions; group changes into projects; define implementation approach
Phase F — Migration Planning	Prioritize projects; create migration roadmap; consider resources, costs, risks, and timelines
Phase G — Implementation Governance	Monitor implementation; check architecture compliance; manage implementation issues
Phase H — Architecture Change Management	Monitor business and technology changes; update the architecture when needed; support continuous improvement
Requirements Management	Operates throughout all ADM phases; tracks, evaluates, and manages changing requirements

FEAF (FEDERAL ENTERPRISE ARCHITECTURE FRAMEWORK)

FEAF — a framework developed by the **U.S. Federal Government** to organize and manage enterprise architecture. Provides a common structure for analyzing the current organization and designing the desired future state. Helps government agencies align business goals, processes, information, and technology. Supports standardization, integration, and better decision-making across agencies.

FEAF Reference Models (MEMORIZE the 6 abbreviations + purpose)

Model	Purpose
PRM — Performance Reference Model	Measures how well an organization and its IT systems achieve strategic goals and desired outcomes
DRM — Data Reference Model	Defines how data is categorized, described, shared, and managed across the enterprise
SRM — Security Reference Model	Defines security-related capabilities, controls, and requirements for protecting enterprise systems and information
BRM — Business Reference Model	Describes the organization's business functions, services, and activities
ARM — Application Reference Model	Organizes IT services and application components that support business functions
TRM — Technical Reference Model	Provides a common structure for technologies, platforms, and technical standards used by the enterprise

GARTNER EA METHODOLOGY

Gartner EA Methodology — developed by **Gartner**, a research and advisory organization. Focuses on **business strategy, stakeholders, and business outcomes** rather than technology alone. Helps organizations understand their current state and define their future state. Emphasizes continuous communication between business and IT.

KEY IDEA: Gartner views Enterprise Architecture as a **continuous process of change and improvement, not a one-time project**.

Gartner EA Process (6 steps)

1. Understand the Business — identify business goals, strategies, and stakeholder needs.
2. Assess the Current State — examine existing processes, systems, data, and technology.
3. Define the Future State — design the desired architecture based on business objectives.
4. Identify Gaps — compare the current and future states to determine what needs to change.
5. Develop & Implement Roadmap — prioritize projects and create a practical plan for achieving the future state.
6. Continuous Improvement — monitor results and update the architecture as business needs change.

CHOOSING THE RIGHT FRAMEWORK (COMPARISON TABLE)

Situation	Suitable Framework
Need to organize and classify EA information	Zachman
Need a detailed EA development process	TOGAF
Government agency or public-sector environment	FEAF
Need flexibility and strong business alignment	Gartner

ENTERPRISE ARCHITECTURE COMPONENT

EA is rarely managed as a single monolithic block. To make complex organizations understandable, governable, and transformable, EA divides the enterprise into distinct, interconnected **domains or components**. Dividing the enterprise into layers follows the principle of **separation of concerns** — letting architects, business leaders, and engineers focus on specific dimensions (business strategy, data flows, application software, hardware infrastructure) while keeping a clear view of how the layers interact and depend on one another.

Most standard EA frameworks (such as TOGAF) categorize the enterprise into **four primary architectural domains: Business, Data, Application, and Technology**.

The Four Core EA Components (Layers) — MEMORIZE

Layer	Definition	Question It Answers	Key Focus Areas
Business Architecture	The foundation of the EA framework; defines the organization's business strategy, governance, organization structure, and key business processes	"What does the business do, and how is it organized to achieve its strategic goals?"	Business strategy, organizational charts, business capabilities, value chains, standard operating procedures
Data (Information) Architecture	The structure and management of an organization's logical and physical data assets and data management resources	"What information does the enterprise need to operate, how is it stored, governed, and how does it flow across systems?"	Data entities, data models (conceptual, logical, physical), master data management (MDM), data governance, information lifecycles
Application Architecture	A blueprint for the individual applications deployment, their interactions, and their relationships to core business processes	"What software applications are needed to automate business processes and manage data?"	Software portfolio management, ERP systems, custom applications, software integration patterns, microservices architecture
Technology Architecture	The hardware, software, and network infrastructure that provides the hosting and runtime environment for applications	"What physical and virtual infrastructure is required to run the application architecture securely and scalably?"	Servers, cloud infrastructure (IaaS/PaaS), operating systems, network topologies, data centers, security infrastructure, end-user hardware

ARCHITECTURE ARTIFACTS

Architecture Artifact — an architectural work product that describes an aspect of the architecture. In simple terms, artifacts are the **deliverables, documents, diagrams, models, matrices, and catalogs** produced throughout the EA lifecycle to capture and communicate architectural designs.

Classification of Artifacts (3 major categories) — MEMORIZE

Category	What It Is	Answers	Examples
Catalogs	A hierarchical list or inventory of things	"What inventory do we have?"	Application Portfolio Catalog, Data Entity Catalog
Matrices	A grid that shows the relationship between two or more architectural elements	"How do elements relate to each other?"	Application-to-Function Matrix, Actor-to-Role Matrix
Diagrams / Models	Visual representations of architecture components and their topology or dynamics	"What does the structure or flow look like?"	Network Diagram, Business Process Flow

Examples of Artifacts Used at Each Layer

Layer	Type	Specific Example	Description & Purpose
Business	Diagram / Model	Business Process Model (BPMN)	Visual flowchart of step-by-step business workflows, decision points, and actors (e.g., Order-to-Cash)
Business	Catalog	Business Capability Map	Hierarchical decomposition of what the business does to fulfill its mission (e.g., Marketing -> Lead Generation -> Campaign Management)
Data	Diagram	Entity-Relationship Diagram (ERD)	Visual model of data entities (e.g., Customer, Order, Product) and relationships/cardinality
Data	Matrix	Information Flow Matrix	Grid mapping which processes/applications create, read, update, or delete (CRUD) specific data entities
Application	Catalog	Application Portfolio Catalog	Comprehensive inventory of all software systems, including version, owner, lifecycle status, and cost
Application	Diagram	Application Communication Diagram	Visual map of how applications integrate, interface, and exchange data (APIs, message queues, batch files)
Technology	Diagram	Network Topology Diagram	Architectural blueprint of routers, firewalls, subnets, zones (DMZ, internal), and cloud connection points
Technology	Catalog	Technology Standards Catalog	Approved hardware, software, and cloud services specifications permitted in the organization

INTEROPERABILITY AND ARTIFACT TRACEABILITY

Key concept — a frequent and critical pitfall in EA is the creation of **siloed or disconnected artifacts** — e.g., polished business process models with no structural or logical connection to the underlying IT software and hardware. When architectural components are isolated, organizations **lose alignment between strategic goals and technical execution**.

Effective EA requires **rigorous traceability across layers**: every technical asset maps directly back to a business need, and every business objective has explicit digital support.

The Anatomy of Cross-Layer Traceability (4 layers) — MEMORIZE

1. **Business Layer (Strategy & Capabilities)** — the chain begins with enterprise strategy; a defined Business Capability (e.g., Customer Support) dictates operational responsibilities. *Artifact*: business capability maps and high-level organizational models.
2. **Data Layer (Information & Entities)** — business capabilities cannot function without information (e.g., customer support requires handling, creating, and modifying data elements). *Artifact*: Conceptual and Logical Entity-Relationship Diagrams (ERDs) identifying critical data entities such as the **Customer Ticket** and its metadata (timestamps, priority levels, resolution status).
3. **Application Layer (Software & Services)** — data must be processed and manipulated by software systems that automate human workflows. *Artifact*: Application Portfolio catalogs and Application Communication diagrams (e.g., enterprise **CRM** or **Zendesk** ticketing platform).
4. **Technology Layer (Infrastructure & Hardware)** — software applications must reside on physical or virtual hardware, operating systems, and network topologies to execute. *Artifact*: Network Topology and Deployment diagrams (e.g., CRM hosted on **AWS EC2** behind secure virtual private clouds / **VPCs**).

Why Traceability Matters: Impact Analysis (MEMORIZE)

- **Proactive Risk Mitigation** — e.g., deprecating an AWS server instance instantly reveals which applications, data entities, and business processes will be disrupted.
 - **Cost Optimization** — leaders identify redundant applications or underutilized infrastructure by tracking them back to the business capabilities they support (or fail to support).
 - **Strategic Agility** — when executive leadership shifts strategy, cross-layer matrices pinpoint exactly which data models, software integrations, and cloud resources need updating.
-

L3: DEVELOPING AN ENTERPRISE ARCHITECTURE

DOCUMENTATION

Key concept — Every organization already has an architecture. Every organization runs on a combination of systems, processes, and infrastructure, regardless of whether any of it has been formally documented. Systems are added over time, by different teams, for different reasons, often without a shared record of how they connect.

The practical consequence: without documentation, decision-makers work from **partial or outdated information** about their own organization.

Why documentation comes before planning

- Plans built without an accurate baseline tend to **underestimate cost, timeline, and complexity**.
- Any structured planning process starts with an accurate assessment of the current situation.
- Organizations cannot reliably plan system upgrades, budget allocations, or process changes without first knowing what already exists.

Documentation and its role

- Systems and processes change regularly, but documentation is often updated far less frequently -> documentation and reality **drift apart**.
- Documentation is frequently treated as a **one-time deliverable** rather than an ongoing responsibility.

Different stakeholders, different needs

- **Technical Staff** — need clear, accurate detail on existing systems before making changes or building integrations.
- **Leadership** — need comparable, reliable information to justify budget and resource decisions.
- **Audit and Compliance** — need a documented record of what changed, when, and why.

CURRENT OPERATIONS AND FUTURE PLANNING

Organizations must maintain what is currently running while planning what comes next. Future planning tends to get more attention and investment than current-state documentation, even though both are necessary.

Current Architectural Views ("As-Is" state) — like taking a complete snapshot of a company's technology and processes right now. *Why it matters:* we need to know exactly what we have before we build or change anything. *The Baseline:* prevents crashing the system when adding new features. *The Blueprint:* gives developers and planners a safe, reliable map to work from.

Keeping track of changes (EACR)

- **Standard Format (EACR)** — companies use an **EA Change Request** to update systems smoothly (like turning in a standardized college paper).
- **Electronic Template** — making forms digital and easy to use encourages stakeholders to participate and follow the rules.
- **The History Book** — archiving these requests creates a detailed history guide for developers to follow during implementation.

The Tech Inventory — EA acts as a giant checklist for all the hardware (computers, servers) and software a company owns.

- **Smart Budgeting** — tells leaders how much money to set aside for maintenance, daily operations, and training.
- **Justifying Upgrades** — provides concrete proof when authorizing new technology purchases.
- **Automated Tracking** — uses barcodes and software systems to keep the list 100% accurate.

Mapping out activities (IDEF-O)

- **Inputs & Outputs** — visual models (like **IDEF-O**) clearly show what data goes in and what comes out.
- **The Rules** — explicitly defines the "**Controls**" (rules to follow) and the "**Mechanisms**" (tools to use).
- **No Guessing** — acts as a strict, foolproof blueprint so developers don't guess when writing code.

Following the data flow — see exactly how a user's request routes through active services (Visualizing Paths); mapping the flow before building prevents accidental cutting of vital connections (Playing it Safe); shows step-by-step how data is consumed, transformed, and outputted (Process Logic).

Connecting the dots — ERDs (Entity-Relationship Diagrams) define how different parts of the business relate (e.g., how a 'Buyer' connects to a 'Listing'). Leaders spot missing links (Finding the Gaps) and mastering the "as-is" state supports strategic decisions.

FUTURE ARCHITECTURAL VIEWS

The **future view** documents the IT resources expected to be active **several years ahead** — as opposed to the "as-is" view, which documents what exists today.

- **Why gaps appear:** without deliberate EA planning, future IT resources drift out of alignment with strategic goals and business services, and duplication builds up. A well-built future view shows exactly how that duplication will be eliminated.
- **Same method, same levels:** future artifacts are built using the same planning assumptions, documentation, and modeling techniques as current-view artifacts — at every level — so change is always traceable.
- **Analogy:** "Having future EA views is like having a **full set of blueprints for the modification of an existing home.**"

Why future architectural views matter (MEMORIZE)

- **Resource Planning** — a coherent view of changing priorities, processes, and resources across the enterprise.
- **Decision-Making Support** — compare current vs. planned states side by side before committing budget or staff.
- **Enterprise-Wide Communication** — a shared "to-be" picture keeps business units and IT teams talking about the same target.
- **Upward & Downward Alignment** — approved future changes stay traceable to strategic goals above and implementation detail below.

DEVELOPING A CONOPS SCENARIO (Concept of Operations)

What a CONOPS scenario documents — good and bad operating environments, drawn from the **SWOT Analysis**, so the enterprise can plan probable **defensive responses and offensive initiatives** in advance.

The Storytelling Technique (Dr. Robert Neilson) — a short story told through the eyes of a central character living through future events, tied to specific planning assumptions in **three areas of change**:

- **Process** — How it gets done
- **People** — Who is involved
- **Technology** — What tools are needed

Stories are easy to remember — and highlighting the assumptions reveals exactly which resources and capabilities the scenario needs.

The "Chess" Analogy — anticipating how an opponent might react to your moves is part of what makes a chess master; top-level play means thinking through several alternative sequences of moves and counter-moves — a dynamic balance between acting and reacting.

FUTURE ARTIFACTS (FIVE LEVELS — MEMORIZE)

Strategic Level — "Directions and priorities"

- Documents the enterprise's overarching goals, direction, and priorities that guide every other level of planning.
- **EA Components:** Strategic Plan; E-Commerce / E-Government Plan.
- **Future-View Artifacts:** Strategic Plan, SWOT Analysis, CONOPS Scenario, CONOPS Diagram, Balanced Scorecard.
- **EXAMPLE:** "Grow e-government service adoption 30% in 3 years" — tracked on the Balanced Scorecard.

Business Level — "Services and processes"

- Documents the products, services, and processes the enterprise uses to carry out its strategic goals.
- **EA Components:** Supply Chains, Business Services, IT Capital Planning Portfolio.
- **Future-View Artifacts:** Business Plan, Node Connectivity Diagram, Swim Lane Process Diagram, Business Process / Service Model, Use Case Narrative & Diagram, Investment Business Case.
- **EXAMPLE:** a Swim Lane Diagram tracing a permit application from Citizen -> Apply -> Portal -> Submit -> Records Office.

Information Level — "Knowledge, systems, databases"

- Documents the data, knowledge, and information flows that support the enterprise's business services.
- **EA Components:** Knowledge Warehouses, Information Systems, Databases.
- **Future-View Artifacts:** Knowledge Mgmt. Plan, Information Exchange Matrix, Logical / Physical Data Model, Activity/Entity (CRUD) Matrix, Data Dictionary / Object Library.

- EXAMPLE: CRUD Matrix entry: 'Permit Record' — Created by Intake, Updated by Reviewer, Deleted by Records Admin.

Systems / Services Level — "Applications and integrations"

- Documents the software applications and services that create, process, and exchange information.
- **EA Components:** Software Applications, Web Services, Service Bus & Middleware, ERP Solutions.
- **Future-View Artifacts:** System Interface Diagram, System Data Flow Diagram, System/Operations Matrix, System Evolution Diagram, Web Application Diagram.
- EXAMPLE: a System Interface Diagram showing the Permit Portal calling a Payment Gateway API.

Infrastructure Level — "Data, telecom, video, cable plant, and security"

- Documents the physical networks, hardware, and facilities — voice, data, and video — that support enterprise systems.
- **Key Future-View Artifacts:** Network Connectivity Diagram, Network Inventory, Capital Equipment Inventory, Building Blueprints, Rack Elevation Diagram.
- Focuses on changes to cable plant, wiring closets, backbone hardware, servers, and remote access resources.
- EXAMPLE: a Network Connectivity Diagram showing a new fiber link between City Hall and the Data Center.

KEEPING FUTURE VIEWS TRUSTWORTHY — FREEZE PERIODS

The Challenge: as changes are implemented, they move from the future view into the current view — so at any moment, what's "current" and what's "planned" can blur.

The Fix — Freeze Periods: freeze the current and future views at regular intervals — e.g., **twice a year**. Changes accumulate for **4–5 months**, then publish as a new release in **month six**.

A BROADER PERSPECTIVE: TOGAF'S VIEW — THREE ARCHITECTURE STATES

TOGAF formalizes the "current vs. future view" as three explicit architecture states:

1. **Baseline Architecture** — the current, "as-is" structure of the enterprise; the starting point for any transformation.
2. **Transition Architecture** — one or more intermediate states that bridge the gap between baseline and target on a realistic timeline.
3. **Target Architecture** — the desired future state: capabilities, processes, and technologies aligned to strategic goals.

Whether it's called a "future view" (EA3 Cube) or a "target architecture" (TOGAF), the core discipline is the same: **document where you're headed with the same rigor used to document where you are**.

ENTERPRISE ARCHITECTURE MANAGEMENT PLAN

EA Management Plan — guides the transition from the current architecture to the desired future architecture; it coordinates IT projects, resources, standards, governance, and implementation activities. **It is a living document that is regularly updated.**

Purpose of the EA Management Plan

- Coordinates multiple IT implementation and upgrade projects.
- Supports prioritization, decision-making, and oversight.
- Connects business strategy with processes, systems, information, and technology.
- Provides a roadmap for moving from the current EA to the future EA.
- Analogy: functions like an architect's project plan for remodeling a house — showing the design, approach, schedule, and sequence of work.

MAIN PARTS OF AN EA MANAGEMENT PLAN (MEMORIZE — 4)

1. **EA Program Management**
2. **Current Architecture Summary**
3. **Future Architecture Summary**
4. **Glossary and References**

Together, these sections explain how the EA program is governed, what currently exists, what must change, and how the plan's terminology is defined.

1. EA PROGRAM MANAGEMENT

Explains how the EA program will be managed.

- **Governance and principles** — establishes policies and decision-making procedures.
- **Strategy and business support** — ensures EA activities support organizational goals.
- **Roles and responsibilities** — defines the duties of sponsors, CIOs, architects, managers, analysts, and other stakeholders.
- **Program budget** — documents annual and long-term costs, including total cost of ownership.
- **Performance measures** — evaluates whether the EA program is effective and efficient.

EA governance, roles, and performance measures

- EA principles may promote information sharing, stakeholder participation, open standards, and the use of IT as a means to achieve business goals.
- The **Chief Architect** manages the EA program, documentation, standards, and configuration management.
- Business and technical architects identify requirements, analyze problems, and design solutions.
- **Performance measures: Outcome measures** (improvements in integration, user satisfaction, or IT investment decisions) and **Output measures** (number of projects completed, scope changes, or planning activities).

2. CURRENT ARCHITECTURE SUMMARY

Describes how existing resources support the organization. It includes: strategic goals and initiatives; business services and information flows; systems and applications; technology infrastructure; IT security; EA standards; workforce skills and training requirements. Helps identify the current role, strengths, weaknesses, and performance gaps of IT.

Current architecture analysis examines: how EA components support strategic goals and business processes; how information moves between departments and systems; whether applications are integrated, scalable, cost-effective, and user-friendly; whether networks and infrastructure reliably support voice, data, video, and mobile services; whether technical standards are followed; what skills and training employees need. NOTE: sensitive security details should be documented in a **separate IT Security Plan** to avoid exposing vulnerabilities.

3. FUTURE ARCHITECTURE SUMMARY

Describes the desired direction of the enterprise.

- **Future operating scenarios** — possible business environments (status quo, aggressive growth, or responding defensively to a market downturn).
- **Planning assumptions** — expected changes in business processes, people, technology, and resources.
- **Updated architecture views** — planned changes documented from strategic goals down to technology components.
- The selected scenario becomes the basis for future EA decisions.

SEQUENCING AND CONFIGURATION MANAGEMENT

- **EA Sequencing Plan** — lists projects, tasks, milestones, dependencies, and timeframes; ensures projects occur in the correct order; prevents problems caused by implementing dependent systems too early; helps coordinate upgrades, migrations, consolidations, and retirements.
- **Configuration Management** — reviews and controls proposed EA changes; applies technical standards; manages waivers and approval deadlines; maintains documentation and version control; uses an **Enterprise Architecture Change Request** reviewed by the Chief Architect and stakeholders.

L3 KEY TAKEAWAYS

- The EA Management Plan is a roadmap for managing enterprise-wide architectural change.
- It links business strategy to IT systems, information, infrastructure, and workforce requirements.
- It documents both current conditions and future goals.
- Sequencing ensures implementation projects are coordinated.
- Configuration management protects consistency, standards, and version control.
- Because the enterprise continuously changes, the plan must be regularly updated and maintained in an EA repository.

L4: USING AN ENTERPRISE ARCHITECTURE

INTRODUCTION

Enterprise Architecture (EA) — a structured approach to organizing an organization's business, data, applications, and technology. It provides a blueprint that shows: how different parts of an organization work together; how technology supports business goals; how systems and processes should be designed; how changes and improvements should be managed.

Why do we need to use EA?

- Without EA: systems become disorganized; departments may use incompatible systems; projects can exceed time and budget; security risks increase; information becomes difficult to manage.
- With EA: better planning; system integration; improved security; reduced duplication; better decision-making; alignment between IT and business goals.

Remember the three areas as a simple sequence: **Plan, Protect, and Manage.**

- **4.1 Planning & Project Management** — projects must be determined by objectives, requirements, resources, schedules, budgets, and potential risks.
- **4.2 Security** — how systems, applications, networks, and information will be protected (access control, data protection, security policies, managing risks).
- **4.3 Repository & Support Tools** — a repository where EA information (diagrams, documentation, standards, policies, system descriptions, models) can be stored and maintained.

REAL-LIFE EXAMPLE: AMAZON — Enterprise Architecture in Action

Amazon's ability to serve millions of customers depends on many interconnected systems.

How one customer order is fulfilled (Enterprise Architecture view):

1. Customer Places Order
2. E-Commerce System
3. Payment System
4. Inventory System
5. Fulfillment System
6. Logistics System
7. Customer Receives Order

Business Goals: Customer obsession, Operational excellence, Innovation, Long-term value.

PLANNING & PROJECT MANAGEMENT

- An EA only creates value when it is executed.
- **Planning — What & Why:** converts the gap between the As-Is and the To-Be architecture into a list of initiatives, and decides which goes first.
- **Project Management — How & When:** controls scope, schedule, cost, quality, resources, and risk so every initiative is delivered as designed.
- **Why it matters:** stops duplicate and siloed systems from being bought and built; ties every peso of IT spending to a business goal; turns the blueprint into a fundable, dated roadmap.

ENTERPRISE ARCHITECTURE PLANNING (EAP)

EAP — the **planning discipline that sits between the architecture and the projects**. Definition (Spewak, 1992): "Defining the architectures needed for the business use of information — and the plan to implement them."

Note the two halves: an architecture AND a plan. Most failed EA efforts produce only the first — plenty of diagrams, no sequence, no budget, no owner.

The Guiding Principle — **MEMORIZE:**

BUSINESS -> DATA -> APPLICATIONS -> TECHNOLOGY

Applications manage data. Technology supports applications. Everything must serve a business process. **EAP is business-driven and data-driven — never technology-driven.** Reverse the order and you get a solution hunting for a problem.

How EAP works — Four Layers

1. **Getting started** — planning initiation: scope, executive sponsor, team, methodology, tools, and a dated workplan. Skipping this is the most common cause of a stalled EA effort.
2. **Where we are today** — business modelling plus a survey of current systems and technology, collected into an **Information Resource Catalog**. This is where the duplicates surface.
3. **Where we want to be** — data architecture first, then applications, then technology. These describe WHAT is needed, not how it will be coded.
4. **How we get there** — the implementation and migration plan: sequence, cost, benefit, resources, and roadmap. This is where architecture finally becomes a budget.

SEQUENCING & PRIORITIZATION — Three More Tests

- **Dependency** — foundations before features: no dashboard without a clean data source.
- **Risk & complexity** — high technical risk? Pilot first, then roll out.
- **Compliance deadline** — non-negotiable work, e.g., the Data Privacy Act.
- **Rule of thumb**: never run two projects that change the same system at the same time.

FROM ARCHITECTURE TO PROJECTS (4 steps — MEMORIZE)

1. **Gap Analysis** — compare the As-Is against the To-Be to find what is missing, duplicated, and obsolete. Every project must trace back to a proven gap.
2. **Work Packages** — group related gaps into small deliverables. Each package gets a clear scope, a single owner, and one concrete outcome.
3. **Business Case** — attach cost, benefit, and risk to each package. Leadership funds outcomes, not diagrams.
4. **Roadmap** — place the packages on a timeline with transition architectures in between, so the business keeps running while it changes.

WHO DOES WHAT (COMPARISON)

Role	Cares about	Asks	Horizon	Delivers
Enterprise Architecture Team	Long-term coherence of the whole enterprise	"Does this fit the target architecture?"	Several years, many projects	Principles, standards, reference models, roadmap
Project Manager / PMO	Delivering this one project	"Are we on scope, schedule, and budget?"	Life of a single project	Project plans, status reports, the working system
Architecture Review Board (ARB)	Checkpoints at initiation, at design, and before go-live	Each review ends in approved / approved with conditions / rejected	Per review	Any deviation needs a documented exception with an expiry date — otherwise the "temporary fix" becomes permanent technical debt

PITFALLS, SUCCESS FACTORS & METRICS (MEMORIZE)

Common Pitfalls (4):

1. **Ivory-tower architecture** — beautiful models nobody follows
2. **No executive sponsor** — the plan becomes a suggestion
3. **Big-bang scope** — everything at once, nothing finished
4. **Stale artifacts** — six months later, nobody trusts them

Success Factors (4):

1. **Active sponsor** — someone who can decide and fund
2. **Small increments** — visible wins every few months
3. **Reviews at project gates** — part of approval, not optional

4. **A living repository** — owned, updated, and used

How we measure it:

- % of projects architecturally compliant
- Redundant apps retired or consolidated
- Delivery time per work package
- Benefits realized vs the business case

SECURITY IN ENTERPRISE ARCHITECTURE

Security means keeping the architecture and everything built on it safe from **unauthorized access, misuse, or damage**. It's part of using the EA, not a separate topic. After the architecture is built, it becomes "live" — real data, real systems, real users. Security should be considered **while implementing the architecture, not after something goes wrong**.

The 4 Main Areas of Security (MEMORIZE):

1. **Data Security** — controlling who can see, change, or take the organization's data.
2. **Network/Technology Security** — protecting servers, connections, and infrastructure.
3. **Application Security** — protecting the software/systems people use daily.
4. **Physical Security** — protecting the actual hardware and facilities.

Controlling who can access what

- **Security policies** — the organization maps out its data, applications, and technology, then puts the plan into use.
- **Role-based access control (RBAC)** — people only get access to what they need for their job (e.g., HR staff can't touch finance data).
- **Authentication** — verifying who a user is before letting them in (passwords, ID checks, etc.).

Security needs planning too — security tasks should be part of the project timeline, not an afterthought (setting rules, assigning access, testing systems); needs teamwork between the EA/IT team and other departments; if security issues are caught too late, it can delay the whole project.

What happens if security is ignored — data can be stolen, leaked, or changed by the wrong people; systems can be disrupted causing downtime and lost work; the organization can lose the trust of employees, clients, or partners.

Key takeaway — Security protects the value of everything the EA was built to do. It's an ongoing job — not something you do once and forget. **A strong architecture is only as good as the security protecting it.**

CORE FUNCTION OF AN EA REPOSITORY

Central Knowledge Base — an **EA Repository** serves as the **master storage facility** for all architectural assets, models, principles, and guidelines across the enterprise. It maintains **single-source-of-truth** visibility for Current (As-Is), Future (To-Be), and Transition states across **Business, Data, Application, and Infrastructure** domains.

Strategic Decision Support — by capturing dependencies between business processes and underlying technology, the repository powers dependency mapping, impact analysis, and gap assessments — helping leadership evaluate downstream technical risk and strategic fit before capital is allocated.

Core Repository Components (3):

1. **Architecture Metamodel** — defines the underlying schema, entity types, taxonomy, and relationship rules governing how architectural artifacts connect to one another.
2. **Architecture Landscape** — catalogs real-time operational views, target blueprints, and active roadmap transitions across all business units and IT domains.
3. **Governance & Standards** — houses regulatory compliance frameworks, technology reference models (TRMs), design principles, and architecture board review logs.

Managing architecture artifacts — Catalogs (structured inventories: application catalogs, business capabilities, server asset lists), Matrices (cross-mapping tables: Application-to-Data CRUD matrix, Role-to-Process matrix), Diagrams (graphical representations of topologies, integration flows, and process models).

KEY CAPABILITIES OF EA SUPPORT TOOLS (4 — MEMORIZE)

1. **Automated Diagramming & Layout** — generates dynamic visual models directly from repository data, eliminating manual drawing and keeping diagrams synchronized with data.

- 2. **Impact & Risk Analysis** — performs traceability queries to evaluate how replacing or modifying an application impacts connected systems and business operations.
- 3. **Collaborative Governance** — provides role-based access control (RBAC), approval workflows, and audit trails for **Architecture Review Board (ARB)** approvals.
- 4. **Automated Data Ingestion** — connects via APIs to cloud platforms, CMDBs, and source repositories to continuously refresh landscape data.

EA TOOLING LANDSCAPE COMPARISON (COMPARISON TABLE)

Support Tool	Primary Focus	Target Environment	Key Strength
LeanIX	Application Portfolio Management	Agile, cloud-native enterprise IT	Fast SaaS setup, automated landscape discovery
Sparx Enterprise Architect	Deep Modeling & System Specs	Complex software & systems engineering	Comprehensive support for TOGAF, ArchiMate, UML, SysML
Software AG Alfabet	Strategic IT Governance & Portfolio	Large enterprise & financial institutions	Robust financial planning, risk & investment alignment
Archi (Open Source)	ArchiMate Modeling	Educational, research, and lean teams	Free, open-source, lightweight graphical modeling

REPOSITORY GOVERNANCE LIFECYCLE (4 STAGES)

- 1. **Data Collection** — generates dynamic visual models directly from repository data, eliminating manual drawing and keeping diagrams synchronized with data.
- 2. **Compliance Check** — performs traceability queries to evaluate how replacing/modifying an application impacts connected systems and business operations.
- 3. **Central Staging** — provides role-based access control (RBAC), approval workflows, and audit trails for Architecture Review Board (ARB) approvals.
- 4. **Continuous Audit** — connects via APIs to cloud platforms, CMDBs, and source repositories to continuously refresh landscape data.

INTEGRATING EA WITH OPERATIONAL IT

SDLC & DevOps Alignment — integrating the EA repository with project management and delivery tools (such as **Jira**, **Confluence**, and **CI/CD pipelines**) ensures developer compliance with approved target architectures. Architectural guardrails are pushed directly into developer workflows, preventing accidental creation of technical debt during sprint cycles.

CMDB & ITSM Synchronization — bi-directional integration with **Configuration Management Databases (CMDBs)** like **ServiceNow** or **AWS Config** reconciles planned architecture with actual runtime reality — providing live verification that deployed physical assets match the strategic models stored in the EA repository.

L5: ALL ABOUT SERVICE-ORIENTED ARCHITECTURE (SOA)

WHAT IS SOA?

SOA (Service-Oriented Architecture) — an approach in software development where we use '**services**' to create our applications.

Service — a software component we use that is assigned a task. We use services in order to make components **reusable and modular**, making developing faster.

What counts as a "service" in software systems: performs a specific function; has a clearly defined interface; is usable by other parts of the system; hides its internal implementation from its consumers; is relatively independent from other services; communicates through a defined method or protocol.

EXAMPLE — online shopping system: User Service (manages user accounts), Product Service (manages products), Order Service (processes orders), Payment Service (handles payments), Shipping Service (handles delivery).

CORE PRINCIPLES OF SOA (4 — MEMORIZE)

1. **REUSABILITY** — use a service in multiple parts of the system (e.g., Website -> Payment Service <- Mobile App). This prevents developers from creating the same functionality multiple times.
2. **LOOSE COUPLING** — proper documentation and description of the service; changes in one service do not affect other components using the service. AWS: "...having as little dependency as possible on external resources ... They should also be **stateless** without retaining any information from past sessions or transactions." Example: an Order Service can request a payment from a Payment Service without knowing how it processes the payment internally.
3. **ABSTRACTION** — customers & clients should not be able to see important logic/code. The app only needs to know how to request a payment, what information to provide, and what response to expect — not the internal logic.
4. **MODULARITY** — SOA divides a large system into smaller, manageable services, each with its own responsibility (e.g., large online shopping system split into User, Product, Order, Payment, Shipping services) — easier to develop, maintain, and modify.

THE MAIN COMPONENTS OF SOA (3 — MEMORIZE)

1. **Service Provider** — creates, maintains, and provides a service (e.g., a company provides a Payment Service that other applications can use).
2. **Service Consumer** — the application or system that requests and uses a service (e.g., an online shopping website requests the Payment Service to process a customer's payment).
3. **Service Registry** — acts as a directory of available services; contains information about what services are available and how consumers can access them.

HOW SOA WORKS

SOA breaks an application into **independent, reusable services** that each handle one business function and talk to each other over a network — rather than living inside one large, tightly-wired program.

How independent services communicate:

- **Synchronous** — the caller sends a request and waits for an immediate reply before continuing — common with REST APIs.
- **Asynchronous / Message-Based** — the caller sends a message and keeps working; a response, if any, arrives later via a queue or bus.
- **Service Discovery** — consumers locate providers at runtime through a service registry (e.g., **UDDI**) instead of a hard-coded address.

COMMON COMMUNICATION PROTOCOLS: REST vs SOAP (COMPARISON)

Aspect	REST	SOAP
Full name	Representational State Transfer	Simple Object Access Protocol
Type	An architectural style, not a strict standard	A strict, formal messaging standard

Aspect	REST	SOAP
Format	Typically JSON (lightweight)	Messages always formatted as XML envelopes
Operations	Uses standard HTTP verbs: GET, POST, PUT, DELETE	Works over HTTP, SMTP, and other transport protocols
State / Security	Stateless — each request contains everything needed to process it	Built-in standards for security (WS-Security) and transactions
Best for	Simpler, faster, widely used for web & mobile APIs	Enterprise systems needing high reliability (banking, finance)

COMMON DATA FORMATS: XML vs JSON (COMPARISON)

Aspect	XML	JSON
Structure	Verbose, tag-based markup	Lightweight, key-value structure
Validation	Strong support for schemas/validation	Maps directly to objects in most programming languages
Usage	Common in SOAP messages & legacy enterprise systems	The default format for REST APIs

THE ENTERPRISE SERVICE BUS (ESB)

ESB (Enterprise Service Bus) — a piece of **middleware** that sits between services, handling routing, translation, and message delivery so that no service ever has to know the technical details of any other service it talks to. Instead of every application connecting directly to every other application, each one only needs to know how to talk to the bus.

Core Functions of an ESB (6 — MEMORIZE):

1. **Routing** — directs each incoming message to the correct destination service based on content or business rules.
2. **Transformation** — converts data formats between systems, such as XML <-> JSON, so mismatched services can still communicate.
3. **Protocol Mediation** — bridges services that use different transport protocols, such as HTTP, JMS, or FTP.
4. **Orchestration** — coordinates calls to multiple services in sequence to complete a single business process.
5. **Monitoring & Logging** — tracks message flow, errors, and performance across the entire system in real time.
6. **Security Enforcement** — applies authentication, authorization, and encryption to messages as they pass through.

Diagram understanding: Web App, Mobile App, CRM System, and Legacy System all connect to the Enterprise Service Bus, which connects to Payment, Inventory, and Shipping Services. All communication flows through the ESB — no service calls another directly.

REAL-WORLD EXAMPLES OF SOA IN APPS (6)

1. **Online Shopping** — separate services for catalog search, cart, payment, and shipment tracking, combined at checkout.
2. **Banking Systems** — one banking app calls independent services for balances, transfers, loan processing, and fraud detection.
3. **Airline Booking** — ticketing, seat selection, payment, and loyalty-point services behind one booking interface.
4. **Streaming Platforms** — recommendation, billing, content-delivery, and user-profile services combine into one app.
5. **Ride-Hailing Apps** — mapping, pricing, driver-matching, and payment services communicate in real time for a single ride.
6. **Healthcare Records** — patient records, billing, and scheduling systems exchange data through shared services.

ANALYZING SOA

Key concept — evaluating SOA requires understanding both its benefits and complexities. While SOA enables rapid scale and iteration, it fundamentally shifts the burden from **code complexity to network complexity**.

Primary Business Benefits (MEMORIZE):

- **AGILITY** — teams can develop, update, and deploy individual services independently -> drastically reduces time to bring new features to market.
- **REUSABILITY** — services are designed as self-contained business functions; a user authentication service can be reused across multiple applications.
- **SCALABILITY** — scale only the specific services under high demand instead of the entire application -> optimizes resource usage and costs.

Technical Advantages:

- **Technology Diversity** — services can be written in entirely different programming languages (polyglot programming) since they communicate over standard protocols (like HTTP).
- **Fault Isolation** — if one service crashes, the rest of the application can often continue to function.
- **Streamlined Maintenance** — smaller codebases make it easier to understand logic, find bugs, and perform maintenance.
- **Independent Deployment** — updates can be pushed to specific services without taking the whole system offline.

Inherent Drawbacks & Challenges:

- **Network Overhead** — heavy reliance on network communication introduces unavoidable latency compared to in-memory function calls; securing data in transit adds complex authentication and encryption challenges.
- **System Complexity** — individual services are simple, but the system as a whole becomes highly complex (dozens/hundreds of independent moving parts); monitoring, debugging, and tracing across service boundaries requires specialized tooling and expertise.
- Anything large scale requires a whole lot of maintenance.

When to choose SOA? Many systems need to connect; services need to be reused; different systems need to share data; old systems need to work with new ones; systems use different technologies; business processes are complex; systems need to work independently.

To sum up (SOA):

- SOA trades code complexity for network complexity, enabling rapid iteration.
- It provides agility through independent deployments, targeted scaling, and fault isolation.
- Challenges include unavoidable network latency, complex security, and difficult system monitoring.
- SOA is ideal when multiple complex or legacy systems must connect and share data.

MONOLITHIC vs SOA (COMPARISON TABLE — MEMORIZE)

Trait	Monolithic Architecture	Service-Oriented Architecture (SOA)
Deployment	Deployed as a single, indivisible unit	Services deployed independently of each other
Technology Stack	Uniform; locked into a single language/framework	Diverse; each service can use the best tool for the job
Scaling	Must duplicate the entire application to scale	Scale only the specific services under heavy load
Failure Impact	A bug in one module can crash the entire system	Failures are isolated to the specific affected service

THE EVOLUTION OF SOA

A Brief History of Software Architecture (MEMORIZE)

Era	Characteristics
MONOLITHIC (1990s)	One codebase, one development. Simple to start, hard to scale or change safely. Every feature ships together; even minor updates require testing and redeploying the entire application
SOA (2000s)	Business logic split into reusable services, integrated over the Enterprise Service Bus. Let large enterprises reuse core capabilities across many applications instead of rebuilding them each time

Era	Characteristics
MICROSERVICES (2010s - now)	Small, independently deployable services communicating over lightweight APIs. Teams build, test, and release each service on its own schedule, using whichever language/database fits best

The Problem That Forced SOA's Invention

- **Scalability bottlenecks** — the entire application had to scale as one unit even if only a single function was under load; modules were so interdependent that a small change could break unrelated functionality.
- **Slow, risky deployments** — a one-line fix meant rebuilding and redeploying the whole system; release cycles stretched to weeks or months.
- **One codebase meant one language, one stack** — migrating to new technology meant rewriting the whole system.

SOA's Answer: Loosely Coupled Services

- **Services, not one app** — business capabilities (billing, inventory, auth) become independent services with defined contracts; each owned, maintained, and scaled by its own dedicated system.
- **Enterprise Service Bus (ESB)** — a central bus routes, transforms, and mediates messages between services — no service calls another directly; also handles logging, security, and protocol translation enterprise-wide.
- **Standardized contracts** — interfaces described with **WSDL/SOAP**, so services on different platforms interoperate (a Java service can talk to a .NET or mainframe service).
- **Reusability across the business** — the same service (e.g., "check credit") can be reused by many applications instead of rebuilt.

Why SOA Evolved Into Microservices

SOA growing pains (4):

- The **ESB became a single point of failure** and a performance bottleneck as message volume grew.
- Heavy **WS-* / SOAP contracts** made services slow to build and change.
- **Centralized governance** slowed teams down (every contract change needed sign-off).
- **Shared services** meant one team's bug or scaling need affected everyone relying on that service.

Microservices' response (4):

- **No central bus** — services call each other directly over lightweight REST/gRPC APIs.
- **Each service owns its data** and can be built in any language or stack best suited to the job.
- **Small, autonomous teams** own a service end-to-end and deploy independently, often many times a day.
- **Failure is isolated** — one service going down doesn't take the whole system with it.

SOA vs MICROSERVICES (COMPARISON TABLE — MEMORIZE)

Aspect	SOA	Microservices
Communication	Enterprise Service Bus (ESB) mediates every message	Direct REST/gRPC calls straight between services
Scope of a service	Broad, enterprise-wide capability, built to be reused widely	Small, single business function per service
Data ownership	Often shared databases across multiple services	Each service owns and manages its own data
Deployment	Coordinated, usually released together as a group	Independent — each service deploys on its own schedule
Governance	Centralized standards & contracts enforced org-wide	Decentralized, owned and evolved by each team

SOA's Legacy in Today's Cloud (4 — MEMORIZE)

1. **API gateways** — the modern successor to the ESB: routing, auth, and rate-limiting for cloud APIs; centralizes cross-cutting concerns like throttling and monitoring.
2. **Service mesh** — tools like **Istio** manage service-to-service communication, echoing SOA's mediation layer; adds retries, encryption, and observability without changing application code.
3. **Containers & orchestration** — **Docker and Kubernetes** let SOA's "independent service" idea run at cloud scale; services packaged once and deployed consistently across any cloud provider.
4. **Hybrid & legacy integration** — cloud platforms still use SOA-style adapters to connect to older enterprise systems (many banks and airlines rely on this pattern to modernize gradually).

L5 KEY TAKEAWAYS

1. Monolithic systems ran into scalability, coupling, and deployment problems as they grew — development slowed and every release carried more risk.
 2. SOA answered this by splitting logic into services connected through a central ESB, letting large organizations reuse services instead of rebuilding logic repeatedly.
 3. SOA's own overhead — the ESB and centralized governance — pushed the industry toward lighter, independently deployable microservices.
 4. SOA's core principles — reusable, contract-based services — live on in today's API gateways, service meshes, and containerized cloud platforms.
-

L6: INTEGRATING TECHNOLOGY

XML FOUNDATIONS

XML (eXtensible Markup Language) — a standard for writing data as plain text, using tags that describe what the data means.

Properties of XML (4):

1. **Self-describing** — tags tell you what each value is.
2. **Extensible** — create your own tags to fit your needs.
3. **Platform Neutral** — plain text; many systems and apps can read it.
4. **Data, not design** — XML stores meaning, not appearance.

EXAMPLE: `<student><name>Jerick Dizon</name><course>BSIT</course><year>3</year></student>` vs normal text "Jerick Dizon, BSIT, 3".

Where XML is used in real life (5):

1. **Microsoft Office files** — a .docx is a ZIP folder full of XML files (text, fonts, formatting all stored as XML).
2. **Android apps** — every app screen layout is written in XML.
3. **RSS feeds and podcasts** — podcast apps download an XML file listing episodes.
4. **Configuration files** — Java, .NET and many systems store settings in XML.
5. **Business documents between companies** — invoices, purchase orders, and bank transactions exchanged as XML files.

Parts of an XML Document (6 — MEMORIZE with example)

Example document:

```
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book isbn="978-0-13-235088">
    <title>Clean Code</title>
    <author>Robert Martin</author>
    <year>2008</year>
  </book>
</library>
```

1. **Prolog** — `<?xml version="1.0" encoding="UTF-8"?>`; always the very first line. version says which XML standard is used; encoding says which character set — UTF-8 lets one file hold English, Filipino, Chinese, and Arabic text.
2. **Root Element** — `<library>...</library>`; the outermost element; there is exactly one per document and everything else sits inside it.
3. **Child Element** — `<book>` is a child of `<library>`; `<title>`, `<author>`, `<year>` are children of `<book>`. An element has three parts: opening tag, content, closing tag.
4. **Attribute** — `isbn="978-0-13-235088"`; always written inside the opening tag; format is `name="value"` and the value must be in quotes; an attribute name cannot repeat on the same element.
5. **Element Content** — the text between the tags (e.g., "Robert Martin"); the actual data being stored.
6. **Nesting** — the parent-child layering that makes XML a tree. Indentation is only for human readability — the computer reads the nesting from the tags.

WELL-FORMED vs VALID (MEMORIZE)

- **Well-formed** (correct syntax): exactly one root element; every opening tag has a closing tag; tags never overlap; attribute values are in quotes; names are case-sensitive.
- **Valid** (well-formed AND follows a schema): only allowed elements appear; required elements are present; values match their declared data types.

Examples:

- NOT well-formed: `<student><name>Ana<age>19</name></age></student>`
- Well-formed but NOT valid: `<student><name>Ana</name><age>banana</age></student>`

- Well-formed and valid: `<student><name>Ana</name><age>19</age></student>`

DTD vs XSD (COMPARISON — MEMORIZE)

Aspect	DTD (Document Type Definition)	XSD (XML Schema Definition)
Status	The old way	Used today
Syntax	Written in its own syntax, not in XML	A W3C standard, written in XML itself
Data types	No data types — everything is just text	Real data types: integer, date, decimal, boolean
Features	No namespace support	Supports ranges, patterns, and lists of allowed values; namespace-aware; types can be reused

XPATH, XQUERY AND XSLT (MEMORIZE the roles)

- XPath** — **FINDS**. Locates one specific part of the document, written like a folder path (e.g., `/library/book/title`). It works like a file path; it doesn't change anything, it just says which part you mean.
- XQuery** — **SEARCHES**. Searches XML and returns only the parts you ask for — what SQL does for tables (e.g., for `$b` in `//book` where `$b/year > 2005` return `$b/title` returns titles of all books published after 2005).
- XSLT** — **CONVERTS**. Turns an XML file into a different format: HTML, plain text, or other XML (e.g., an `<xsl:template match="book">` turns each `<book>` into an HTML list item). Same data, different output — the original file is never changed.

READING XML IN A PROGRAM (4 approaches — MEMORIZE)

Approach	How It Works	Best For
DOM	Loads the whole document into memory as a tree	Easy to read and edit; best for small files you need to modify (uses memory equal to the file size)
SAX	The parser streams through and fires events at your code	Very little memory; read once, forward only, cannot edit; best for very large files
StAX	Your code asks for the next piece when ready	Same efficiency as SAX; you control the loop; best for streaming where you need to skip sections
Data Binding (JAXB)	Converts XML directly into objects (e.g., <code><student></code> becomes a student object with <code>getName()</code>)	Best for when the schema is stable

TREES MEET TABLES — THE IMPEDANCE MISMATCH

- XML is a **nested, ordered tree**; a relational database is a **flat table**. Storing one in the other is never free.
- Order matters in XML**; table rows have no order.
- Elements repeat or disappear** — one order has 1 item, another has 50; a table can't have 50 columns.
- Mixed content** has no column equivalent.
- The gap between the XML tree and database tables is the **impedance mismatch**.

2 KINDS OF XML, 3 WAYS TO STORE (MEMORIZE)

First, classify the document:

- Data-Centric** — regular, predictable structure; order doesn't matter (invoices, purchase orders, config files) -> store in a **relational database**.
- Document-Centric** — irregular, mixed content; order matters (manuals, contracts, articles) -> store in a **native XML database**.

Second, choose storage approach (3):

1. **XML-enabled database** — a normal relational database with XML features added (Oracle XML DB, SQL Server, Db2 pureXML). *Gain:* speed and familiar tools. *Cost:* the original document is lost.
2. **Native XML database** — the document itself is the unit of storage (eXist-db, BaseX, MarkLogic). *Gain:* perfect fidelity, flexible structure. *Cost:* slower joins, fewer experts.
3. **Hybrid / XML column** — keep the whole document in one column, copy key fields into normal columns. *Gain:* both fidelity and speed. *Cost:* data stored twice, must stay in sync.

Three internal storage techniques:

- **Shredding** — the tree is broken into rows and columns. Fast joins and reports, but the original document no longer exists.
- **CLOB / text column** — stored as one long string. Perfect fidelity and cheap writes, but every query must parse it again.
- **Native tree** — parsed once into an indexed form. Fast XPath, but larger on disk and vendor-specific.

XML IN PRACTICE

XML Integration — the strategic practice of using XML as a **universal, text-based standard** to exchange structured business data between incompatible enterprise systems, databases, and external business partners without redesigning their core architectures.

- **The Mission** — bridges isolated enterprise silos (ERP, CRM, legacy mainframes) across conflicting operating systems and programming languages.
- **The "Lingua Franca"** — acts as a vendor-neutral, self-describing common language with explicit tags that any hardware or software platform can interpret.
- **Enterprise Impact** — forms the essential data payload for **Enterprise Application Integration (EAI)**, B2B supply chains, and automated business workflows.

The Integration Problem & Solution:

- *Problem* — **Enterprise Silos**: companies rely on multiple distinct systems (ERPs like SAP, CRMs, legacy databases, custom logistics apps); each runs on different OS, protocols, and proprietary formats; traditional **point-to-point connections** create an unmaintainable "spaghetti" mesh.
- *Solution* — **Universal XML Lingua Franca**: platform independence, self-describing data, and decoupled architecture — systems send canonical XML payloads to middleware or service buses rather than binding directly to each other.

How XML powers integration (3):

1. **XSLT Transformation** — bridges differing schemas without modifying original databases: converts supplier PO formats to canonical enterprise standards; maps incompatible field names automatically; transforms raw XML into HTML or JSON.
2. **Schema Validation (XSD)** — enforces strict structural rules and data integrity at the system boundary: guarantees required fields and correct data types; rejects corrupted or malformed payloads early; prevents invalid records from polluting databases.
3. **Standardized Formats** — enables global B2B collaboration through shared domain vocabularies: - **ISO 20022** — global interbank payments - **HL7 / CDA** — healthcare & clinical data - **RosettaNet / ebXML** — B2B supply chains

Summary of XML Integration: XML transforms isolated silos into a cohesive enterprise data network. XSD protects data integrity; XSLT resolves schema mismatches. Governed, high-stakes industries (banking, healthcare) still rely on XML for certified compliance.

WEB SERVICE TECHNOLOGIES

Web service technologies — standards, protocols, and formats that allow different software applications to communicate and exchange information over a network. They allow systems developed using different programming languages, operating systems, and platforms to work together (e.g., a mobile app communicating with a web server for user or product data).

SOAP (Simple Object Access Protocol) — a protocol used for exchanging structured information between applications over a network. Traditionally uses XML for messages and follows strict rules.

- **Advantages:** standardized communication structure; supports enterprise-level security and transactions; reliable for complex business processes; works across different platforms.
- **Disadvantages:** more complex to implement; messages can be relatively large; requires more processing than simpler approaches.

REST (Representational State Transfer) — an architectural style used to design web-based services and APIs. Commonly uses standard HTTP methods; simpler and more lightweight than SOAP.

- **Advantages:** simple and easier to implement; lightweight; works well with web and mobile applications; scales well; commonly uses JSON.
- **Disadvantages:** does not provide the same level of built-in standards as SOAP; security mechanisms usually implemented separately; may not be ideal for complex enterprise transactions.

XML (Extensible Markup Language) — a format used to organize and exchange structured information; became especially important in traditional web services because SOAP uses it to structure its messages. *Characteristics:* human-readable; platform-independent; can represent complex data; customizable tags; commonly associated with SOAP services.

JSON (JavaScript Object Notation) — a lightweight format for exchanging data; especially common in REST APIs and modern web/mobile applications; generally more compact and easier to process than XML. *Characteristics:* lightweight; easy to read; easy to process; commonly used with REST; widely supported by programming languages.

HTTP and HTTPS — **HTTP (Hypertext Transfer Protocol)** is the primary communication protocol used on the Web, commonly used by REST APIs; a client sends requests to a server and receives responses. **HTTPS (HTTP Secure)** is the secure version — it uses **TLS encryption** to protect information in transit, important when handling sensitive information (passwords, payment info, personal data).

WSDL (Web Services Description Language) — an XML-based language traditionally used to describe SOAP web services.

Key concept: WSDL acts like a **technical contract or specification** for a SOAP service — it provides: what services are available, what operations the service supports, what information can be sent/received, and how the service can be accessed.

UDDI (Universal Description, Discovery, and Integration) — created to help organizations publish and discover web services; functioned like a directory where businesses and applications could find information about available services. Important in the early development of web services but less commonly used in modern web development today.

Importance of web service technologies (5): Interoperability (different systems can communicate), Integration (separate applications work together), Reusability (same service used by multiple applications), Scalability (support growing numbers of users/applications), Automation (systems exchange information automatically).

ADVANCED WEB SERVICES — WS-* TECHNOLOGIES

WS-* (WS- Technologies) — a **family of specifications built on top of SOAP and XML**, designed to support enterprise-grade web services. They are **composable** — built using XML, WSDL, and SOAP extensibility models. While basic SOAP handles message exchange, it **doesn't address security, reliability, or transaction management on its own** — that is the gap the WS- family fills.

Core WS- Specifications:

- **WSDL** — an XML-based service description language that complements UDDI by providing a uniform way to describe services and their network protocol bindings.
- **UDDI** — provides an open, platform-independent framework for describing services, discovering businesses, and integrating services over the internet.
- **WS-Security** — adds **authentication, encryption, and message integrity** to SOAP messages; often paired with security tokens like **SAML assertions**, which help prevent man-in-the-middle and replay attacks.
- **WS-ReliableMessaging** — ensures messages are still delivered correctly even over unstable networks.

WS-* in practice — these specifications rarely work in isolation; most systems combine several at once. WS-Policy expressions can be embedded into a WSDL file; **WS-SecurityPolicy** can define concrete scenarios (e.g., requiring a username token with a plain text password for authentication). This layering keeps enterprise systems both flexible and secure at scale.

WEB SERVICES INTEGRATION

Web services integration — the process of connecting multiple independent services or systems so they function together as one cohesive whole — necessary because organizations rely on a mix of platforms, languages, and legacy systems never designed to work together directly.

Integration approaches and patterns:

- **Orchestration** — relies on a **central process** that directs how different services interact, coordinating them from the top down.
- **Choreography** — services communicate directly with one another in a **peer-to-peer** manner, with no single controlling process.

- **Middleware** — many enterprise systems rely on middleware, such as an **Enterprise Service Bus**, to manage communication between services.
 - Web services standards such as **SOAP, WSDL, UDDI, and WS-I profiles** make cross-platform integration technically possible.
-

L7: INTEGRATING APPLICATIONS

APPLICATION INTEGRATION

Application Integration — the process of enabling independently designed applications — often built by different vendors, using different technologies, platforms, or data structures — to communicate, exchange data, and work together as a coordinated system.

Goal: create a seamless flow of information and processes across systems, reducing or eliminating manual data transfer.

Why it's needed: organizations use specialized applications such as CRM, ERP, HR, and accounting systems that operate independently and store data differently.

Why application integration matters:

- **Eliminates data silos** — connects isolated data across departments (sales, HR, finance) so teams access consistent, up-to-date information.
- **Reduces manual data entry and human error** — automates data flow, freeing staff for higher-value work.
- **Improves data accuracy and consistency** — keeps records aligned across systems (an update in one place is reflected elsewhere).
- **Speeds up decision-making** — provides leadership with integrated, up-to-date data.

INTEGRATION STYLES (MEMORIZE each definition)

POINT-TO-POINT INTEGRATION — a direct, custom-built connection between two systems, built specifically for that pair, generally not reusable.

- Characteristics: simple and fast to implement for a small number of systems.
- Limitation: becomes difficult to manage and scale as more systems are added — leads to a tangled "spaghetti" architecture.
- Example: a website directly connects to a payment gateway without an intermediary.

HUB-AND-SPOKE INTEGRATION — a central hub (integration engine) manages and routes communication between multiple systems, handling data translation and routing.

- How it works: each system connects to the hub, reducing the total number of connections.
- Benefit: easier to manage and scale than point-to-point.
- Example: a central integration hub connects HR, payroll, and benefits systems.

ENTERPRISE SERVICE BUS (ESB) — a middleware layer that facilitates, transforms, and manages communication between multiple applications.

- How it works: acts as a common "communication backbone," routing and transforming messages between systems using different formats and protocols.
- Benefit: centralized integration capabilities, easier to manage and scale in larger environments.
- Example: a bank uses an ESB to connect core banking, mobile app, and ATM network (XML and JSON).

API-BASED / MICROSERVICES INTEGRATION — a modern, flexible approach where applications or services expose specific functions through APIs that other systems can call.

- How it works: independent components communicate through well-defined APIs; components can be updated, replaced, or scaled without changing the whole system.
- Benefit: highly scalable and flexible; widely used in cloud, web, and mobile apps.
- Example: a ride-sharing app uses APIs to integrate Google Maps (navigation), Stripe (payments), and Twilio (SMS).

REAL-TIME vs BATCH INTEGRATION (COMPARISON)

Aspect	Real-time integration	Batch integration
Timing	Data synced immediately or with very little delay	Data collected and transferred at scheduled intervals

Aspect	Real-time integration	Batch integration
Best for	Immediacy is critical (e.g., stock trading platform updates balances right after every trade)	Large volumes that don't need immediate processing (e.g., payroll syncs attendance nightly)
Advantage	Systems reflect the most current information	Reduces processing overhead and system load

Common challenges in application integration: data format mismatches (requires transformation), security risks (more entry points for attacks), system downtime/dependency (one failure disrupts dependents), complexity as systems grow (more maintenance and points of failure).

LEGACY SYSTEM INTEGRATION

Legacy system integration — the process of connecting and enabling communication between older, often outdated systems (legacy systems) and newer systems or technologies — essential for legacy systems to keep functioning within a modern IT environment.

Core approaches (5): API Wrapping, Middleware Connectivity, Enterprise Service Bus, Integration Platform as a Service (iPaaS), ETL & Data Synchronization.

Why do businesses still use them?

- **Cost savings** — avoids the massive expense and risk of rebuilding core databases.
- **Better efficiency** — stops slow, manual data copying.
- **New features** — adds modern tools (cloud analytics, automation, AI) on top of older systems.
- **Unified data** — different teams share a single, accurate view.
- **Faster results** — updates workflows quickly instead of waiting years for full replacement.

COMPARISON: Legacy Integration vs Implementing New Systems (COMPARISON TABLE)

Feature	Legacy System Integration	New System Implementation
Cost	Lower upfront by building on existing infrastructure	Higher investment for more capability
Disruption	Minimal disruption to daily workflows and operations	High risk of operational downtime during the transition
Training	Minimal training needed — staff already know the system	Extensive training required for new interfaces and workflows
Scalability	Limited by the architectural ceiling of the old system	Highly scalable and easily customizable for future growth

Challenges of integrating legacy systems (6):

1. **Architectural Mismatch** — older monolithic designs struggle to communicate with modern, API-driven or microservices-based cloud environments.
2. **Data Inconsistency** — legacy databases use proprietary formats requiring complex translation logic.
3. **Security Gaps** — outdated encryption and authentication protocols create vulnerabilities.
4. **Skill Shortages** — finding engineers who understand obsolete languages is increasingly difficult.
5. **Missing Documentation** — poor records make troubleshooting and integration risky.
6. **Downtime Risks** — connecting/updating these systems often forces costly operational pauses or crashes.

Best practices (focus on) — define clear business outcomes, map dependencies, and adopt incremental patterns like the **Strangler Fig pattern** to minimize operational risk.

- **Strategic Planning & Assessment:** Define Business Outcomes; Map all dependencies; Prioritize by impact.
- **Architectural & Technical Patterns:** Use incremental approaches; Employing **Anti-Corruption Layer (ACL)**; Leverage modern middleware.
- **Data & Testing Operations:** Treat data as separate workstream; Automate testing; Ensure robust monitoring.

ENTERPRISE INTEGRATION

Enterprise Integration (Enterprise Application Integration / EAI) — the process of linking separate applications, databases, and hardware platforms across an entire organization to work together as a coordinated ecosystem.

Goal: create a seamless, automated flow of information and end-to-end processes across corporate systems, eliminating data isolation and manual work.

Key objectives (4):

1. **Data Consistency** — synchronizes records across departments to establish a reliable, single source of truth.
2. **System Interoperability** — bridges legacy software, on-premises infrastructure, and modern cloud services using standardized communication protocols.
3. **Business Process Automation** — automates complex multi-system workflows (e.g., order-to-cash, employee onboarding).
4. **Real-Time Visibility** — delivers integrated, real-time analytics and cross-functional reporting for executive decision-making.

Enterprise integration architectures (4):

- **Enterprise Service Bus (ESB)** — a centralized middleware backbone managing routing, message transformation, and protocol translation.
- **Hub-and-Spoke Model** — a central broker routes traffic and translates data between connected "spoke" applications, reducing total connections.
- **Service-Oriented Architecture (SOA)** — applications expose core capabilities as reusable, loosely coupled services across a network.
- **Event-Driven Architecture (EDA)** — systems publish event messages to an enterprise broker, letting subscribing services react asynchronously in near-real time.

Core enterprise integration patterns (4):

- **Messaging Channels** — dedicated pathways for applications to send and receive asynchronous data payloads.
- **Content-Based Router** — inspects incoming message contents and automatically routes data to the correct destination based on business rules.
- **Message Translator** — converts data payloads between incompatible structures and formats (e.g., XML to JSON, EDI to SQL).
- **Publish-Subscribe Channel** — delivers messages from a single sender to multiple subscribed systems simultaneously without direct coupling.

Business & operational benefits (4): Operational Efficiency (cuts manual entry/error), Organizational Agility (adopt/replace tools without rebuilding IT infrastructure), Scalability (growing volumes through organized middleware), Cost Optimization (preserves and extends legacy investments).

Challenges in enterprise integration (4): High Architectural Complexity, Single Point of Failure (centralized hubs/ESBs), Data Security & Compliance, Resource & Skill Requirements.

Best practices (4): Enforce Loose Coupling; Adopt API Standardization; Implement Centralized Monitoring; Design for High Availability (failover, load balancing, message retry queues).

INTEGRATION STRATEGIES

Integration strategy — determines how systems should communicate, where integration logic should live, and how the environment can scale and remain maintainable (Architecture, Data Flow, Messaging, APIs & Events).

Common integration strategies (5):

- **Point-to-Point** — direct connection between two systems; simple for a small number of integrations, difficult to manage at scale.
- **Hub-and-Spoke** — systems connect through a central hub that routes and transforms communication.
- **ESB** — middleware acts as a communication backbone for routing, transformation, and service interactions.
- **API-Based** — systems expose reusable functions and data through defined APIs.
- **Event-Driven** — systems react to published events, supporting asynchronous, near-real-time flows.

POINT-TO-POINT vs CENTRALIZED INTEGRATION (COMPARISON TABLE)

Key trade-off: **simplicity at small scale versus centralized control and manageability as the environment grows.**

Aspect	Point-to-Point	Hub-and-Spoke / ESB
Setup	Fast for a few connections	More planning; centralized integration layer
Management	Every connection managed separately	Central routing and policies improve visibility
Scalability	Connection count grows quickly	New systems can often connect through the hub
Best fit	Small/simple environments	Larger, more complex enterprise environments

API-based integration — APIs provide defined interfaces that allow applications to request data or functions without depending on internal implementation details. Mobile/web app requests a service -> API layer authenticates, routes, and defines the interface -> back-end provides data/functionality. **Reusable** (same API serves multiple clients), **Controlled** (auth at the interface), **Flexible** (back-ends evolve without exposing internals).

Event-driven integration — instead of one system directly calling every other, a system **publishes an event** that interested consumers react to (Event Producer e.g., "Order Created" -> Event Broker publishes/routes -> Consumers: Inventory, Payment, Notification). Provides **loose coupling**, **asynchronous flow**, **near-real-time response** — useful when many services may need to react to the same business event.

Cloud & iPaaS integration — **iPaaS (Integration Platform as a Service)** provides cloud-hosted tools for connecting applications and data across cloud and on-premises environments (on-premises legacy apps/databases <-> iPaaS connect/transform/monitor <-> cloud SaaS, cloud databases, analytics). Benefits: faster connection setup (reusable connectors), hybrid environments, centralized monitoring.

Choosing the right integration strategy — no single strategy fits every organization; match to business needs, technical constraints, and future growth:

- **Number of systems** — few systems tolerate direct connections; many need stronger coordination.
- **Real-time needs** — critical immediate updates favor APIs or event-driven flows.
- **Data volume** — large scheduled transfers suit batch-oriented approaches.
- **Security** — sensitive information needs strong authentication, authorization, monitoring.
- **Scalability** — choose patterns that remain manageable as integrations increase.
- **Budget & skills** — consider platform cost, maintenance effort, and available expertise.

L8: ENTERPRISE INTEGRATION

ENTERPRISE INTEGRATION

Enterprise Integration — the process of **linking different IT systems, applications, and data sources so they can work together and share data seamlessly**. Goal: make separate systems (e.g., HR, sales, inventory, finance) function as one unified system. **Key idea: it's about connecting, not replacing, existing systems.**

Why it matters: companies use many systems that don't "talk" to each other -> data silos, duplicate entries, inconsistent records. Integration removes these gaps. Benefits: real-time data sharing, faster decisions, reduced manual work/errors, cost savings, better customer service.

The integration challenge: enterprises use systems built on different platforms, languages, and data formats (old legacy systems vs modern cloud apps) — e.g., a sales system in one format, an accounting system in another. This mismatch is why common standards (like XML) and communication methods (like Web Services) were developed.

Approaches to integration: Point-to-Point (direct connections; simple but hard to scale); Hub-and-Spoke (central hub manages connections); Enterprise Service Bus (middleware that lets many systems communicate through one channel); API/Web Services-based (modern approach using standardized calls).

Key technologies overview:

- **XML** — a standard format for structuring and exchanging data between different systems, regardless of platform.
- **Web Services** — standardized methods (e.g., SOAP, REST) that let applications communicate over a network.
- **SOA (Service-Oriented Architecture)** — a design approach where business functions are built as reusable "services" that can be combined and reused across the enterprise.

XML INTEGRATION

XML Integration — the practice of exchanging structured data between systems **using XML as a common format** — reliable, consistent, secure, and fast.

Why best practices matter (failure modes — MEMORIZE):

- **Inconsistent data** — same field, different tags across systems -> mapping breaks silently.
- **Invalid information** — missing or malformed values corrupt downstream records.
- **Security risk** — unvalidated input opens the door to injection and disclosure.
- **Slow processing** — bloated payloads and deep nesting drag down throughput.
- **Hard troubleshooting** — vague failures with no context waste hours per incident.
- **Compatibility issues** — undocumented schema changes quietly break trading partners.

Clear structure & data consistency:

- **Structure:** meaningful names (e.g., <shippingAddress>, not <sa>); logical hierarchy (order -> customer -> contact); flat & DRY (avoid unnecessary nesting; never duplicate a value).
- **Consistency:** same tag every time (one name per concept, applied identically); consistent types & formats (quantity always a number; dates always ISO 8601); define required fields (mark what must be present).

Validate & secure the exchange:

- **Validation — layered checks:** well-formed syntax; schema (XSD) check; matching tags & root; required fields & types.
- **Security — protect every hop:** use HTTPS (encrypt XML in transit); never trust input (validate against schema and business rules); guard sensitive data (mask/encrypt PII at rest and in tags); authenticate (tokens, certs, or API keys); least privilege; block XML attacks (disable external entities; cap size/depth).

Optimize performance & handle errors:

- **Performance:** send only necessary data; keep XML reasonably sized (paginate/stream large collections; gzip when useful); process data efficiently (prefer streaming/SAX over full DOM for large files).
- **Error handling:** detect & identify (catch the failure where it occurs, pin down the root cause); give clear feedback (specific, actionable message — not a generic error code).

Best practices, compounded — what you gain: reliable integrations, faster processing, accurate data exchange, easier troubleshooting, better security, better compatibility. **Design for the reader** (structure, name, document XML as if a new engineer must understand it cold); **trust nothing by default** (well-formed is not the same as safe); **optimize the whole lifecycle** (lean

payloads, clear errors, living documentation).

WEB SERVICES

What are web services? — a standardized way for applications to communicate over a network, regardless of platform or language. Platform-independent (works across Windows, Linux, mobile); protocols used: HTTP/HTTPS (sometimes SMTP/FTP); data formats: XML or JSON; loosely coupled. Examples: payment gateways, courier tracking, social media login.

APIs: the gateway to web services — an **API (Application Programming Interface)** defines how software components should interact.

- **REST APIs** — most common today; used in web & mobile apps.
- **SOAP APIs** — enterprise-grade; used in banking/finance.
- **GraphQL** — flexible querying; request only what you need.
- APIs act as a "**contract**" — defines available requests and expected responses; good APIs are well-documented (e.g., Swagger/OpenAPI). Real-world examples: Google Maps API, Lalamove/J&T API, Stripe API.

SOAP vs REST (COMPARISON TABLE — MEMORIZE)

Feature	SOAP	REST
Type	Protocol	Architectural style
Format	XML only	JSON, XML, plain text
Security	Built-in (WS-Security)	Depends on implementation
Speed	Slower (heavier messages)	Faster (lightweight)
Best for	Banking, enterprise, government	Web & mobile apps
Example	Interbank transfer systems	Facebook, Google Maps API

Security & authentication:

- **Security:** HTTPS/SSL/TLS encryption (protects data in transit); input validation (prevents SQL Injection, XSS); WS-Security for SOAP (message-level security); rate limiting & firewalls (prevents abuse/DDoS); data encryption at rest.
- **Authentication:** API Keys, OAuth 2.0, JWT; Multi-Factor Authentication (MFA).
- Remember: **Authentication = "who are you?" vs Authorization = "what can you do?"**

Reliability & performance:

- **Reliability:** redundancy & failover systems; retry mechanisms; message queuing (RabbitMQ, Kafka); health monitoring & SLA (e.g., 99.9% uptime guarantee).
- **Performance:** caching; load balancing; data compression (GZIP) & pagination; CDN (serves content closer to the user).

Web services best practices — checklist: use HTTPS always; choose the right protocol (SOAP vs REST); implement proper authentication (OAuth, JWT, MFA); validate all inputs; design for failure (redundancy, retries, failover); optimize with caching, load balancing, CDNs; document your API clearly (Swagger/OpenAPI); version your APIs (e.g., /api/v1/); monitor performance and uptime continuously.

SERVICE-ORIENTED ENTERPRISE (SOE)

Service-Oriented Enterprise (SOE) — connects business systems through **reusable services**. It allows different applications and departments to communicate and share data. It focuses on **integration, flexibility, and reuse of existing systems**. Example: a school connects its student information system, payment system, library system, and learning platform through services.

Service-Oriented Architecture (SOA) — an approach that organizes enterprise systems into independent, reusable services. Services communicate through standard interfaces and protocols. Key characteristics: **loose coupling, reusability, interoperability, and scalability**.

Main components of a Service-Oriented Enterprise (5 — MEMORIZE):

1. **Service Provider** — creates and maintains a service.
2. **Service Consumer** — application or user that uses the service.
3. **Service Registry / Broker** — helps consumers find available services.

4. **Service Interface** — defines how a service can be accessed.
5. **Enterprise Service Bus (ESB)** — connects and manages communication between services.

Building the Service-Oriented Enterprise (6 steps):

1. Identify business processes that need integration.
2. Identify existing systems and services that can be reused.
3. Design services based on specific business functions.
4. Create standard interfaces for communication.
5. Integrate, test, and monitor the services.
6. Maintain and improve services as business needs change.

Benefits and Challenges:

- **Benefits:** better system integration and flexibility; reuse of services reduces development time and costs; easier maintenance and expansion.
- **Challenges:** security and access control; complex integration between different systems; requires proper service management and monitoring.

Summary: a Service-Oriented Enterprise connects business systems through reusable services; SOA provides the structure for designing and integrating these services; key components include providers, consumers, interfaces, registries, and ESBs. The goal is an enterprise that is **integrated, flexible, reusable, and easier to manage**.

IDENTIFICATION BANK

Match the exact wording you may see in the exam.

L2 Identification

A detailed, step-by-step approach that describes how an Enterprise Architecture program is established, developed, managed, maintained, and used within an organization. -> **EA Implementation Methodology**

A structured set of tools, models, and guidelines used to organize and design an enterprise's architecture. -> **EA Framework**

Developed by John Zachman in 1987; not a methodology but a classification schema organized as a 6x6 matrix. -> **Zachman Framework**

An EA framework used to help organizations design, plan, implement, and manage their enterprise architecture. -> **TOGAF**
TOGAF's method that helps organizations identify business requirements, analyze the current architecture, design the target architecture, identify changes, plan implementation, and manage/review the architecture. -> **ADM (Architecture Development Method)**

The ADM phase that establishes EA principles and governance and defines roles, responsibilities, and processes. -> **Preliminary Phase — Preparation**

The ADM phase that defines goals and scope, identifies stakeholders, and creates the overall architecture vision. -> **Phase A — Architecture Vision**

The ADM phase that defines infrastructure, networks, platforms, and technologies. -> **Phase D — Technology Architecture**

The ADM element that operates throughout all phases, tracking and managing changing requirements. -> **Requirements Management**

A framework developed by the U.S. Federal Government to organize and manage enterprise architecture. -> **FEAF**

The reference model that measures how well an organization and its IT systems achieve strategic goals and desired outcomes. -> **PRM (Performance Reference Model)**

The reference model that defines how data is categorized, described, shared, and managed across the enterprise. -> **DRM (Data Reference Model)**

The reference model that defines security-related capabilities, controls, and requirements for protecting enterprise systems and information. -> **SRM (Security Reference Model)**

The reference model that describes the organization's business functions, services, and activities. -> **BRM (Business Reference Model)**

The reference model that organizes IT services and application components that support business functions. -> **ARM (Application Reference Model)**

The reference model that provides a common structure for technologies, platforms, and technical standards used by the enterprise. -> **TRM (Technical Reference Model)**

An EA methodology developed by Gartner that views Enterprise Architecture as a continuous process of change and improvement, focused on business outcomes rather than technology alone. -> **Gartner EA Methodology**

The foundation of the EA framework; defines the organization's business strategy, governance, organization structure, and key business processes. -> **Business Architecture**

The structure and management of an organization's logical and physical data assets and data management resources. -> **Data (Information) Architecture**

A blueprint for the individual applications deployment, their interactions, and their relationships to core business processes. -> **Application Architecture**

The hardware, software, and network infrastructure that provides the hosting and runtime environment for applications. -> **Technology Architecture**

An architectural work product that describes an aspect of the architecture — the deliverables, documents, diagrams, models, matrices, and catalogs produced throughout the EA lifecycle. -> **Architecture Artifact**

A hierarchical list or inventory of things that answers "What inventory do we have?" -> **Catalog**

A grid that shows the relationship between two or more architectural elements that answers "How do elements relate to each other?" -> **Matrix**

Visual representations of architecture components and their topology or dynamics that answer "What does the structure or flow look like?" -> **Diagram / Model**

The pitfall where architectural components are isolated and alignment is lost between strategic goals and technical execution.

-> **Siloed / disconnected artifacts**

The four domains into which most standard EA frameworks (such as TOGAF) categorize the enterprise. -> **Business, Data, Application, Technology**

L3 Identification

The formal request companies use to update systems smoothly, archiving a history for developers to follow during implementation. -> **EA Change Request (EACR)**

A visual modeling technique that clearly shows the Inputs & Outputs, the "Controls" (rules), and the "Mechanisms" (tools) of a process. -> **IDEF-O**

Diagrams that define how different parts of the business relate to each other (e.g., how a 'Buyer' connects to a 'Listing'). -> **ERD (Entity-Relationship Diagram)**

A scenario document that covers good and bad operating environments, drawn from the SWOT Analysis, to plan defensive responses and offensive initiatives in advance. -> **CONOPS Scenario (Concept of Operations)**

A short story told through the eyes of a central character living through future events, tied to planning assumptions in process, people, and technology. -> **The Storytelling Technique (Dr. Robert Neilson)**

The three areas of change in the CONOPS storytelling technique. -> **Process (How), People (Who), Technology (What tools)**

The technique of 'freezing' the current and future views at regular intervals (e.g., twice a year), publishing changes as a new release in month six. -> **Freeze Periods**

The current, "as-is" structure of the enterprise — the starting point for any transformation. -> **Baseline Architecture**

One or more intermediate states that bridge the gap between baseline and target on a realistic timeline. -> **Transition Architecture**

The desired future state — capabilities, processes, and technologies aligned to strategic goals. -> **Target Architecture**

The plan that guides the transition from the current architecture to the desired future architecture; a living document that coordinates IT projects, resources, standards, governance, and implementation. -> **EA Management Plan**

The four main parts of an EA Management Plan. -> **EA Program Management, Current Architecture Summary, Future Architecture Summary, Glossary and References**

The person who manages the EA program, documentation, standards, and configuration management. -> **Chief Architect**
Improvements in integration, user satisfaction, or IT investment decisions (a type of performance measure). -> **Outcome measures**

Number of projects completed, scope changes, or planning activities (a type of performance measure). -> **Output measures**

The future-view artifact used to track "Grow e-government service adoption 30% in 3 years." -> **Balanced Scorecard**

The tool used to trace watertight reasoning from Citizen -> Apply -> Portal -> Submit -> Records Office. -> **Swim Lane (Process) Diagram**

The level of future artifacts that document the physical networks, hardware, and facilities — voice, data, and video. -> **Infrastructure Level**

The level of future artifacts that documents lifecycle management of a Permit Record with CRUD operations. -> **Information Level**

L4 Identification

A structured approach to organizing an organization's business, data, applications, and technology. -> **Enterprise Architecture (EA)**

The simple sequence that summarizes the three areas for using an Enterprise Architecture. -> **Plan, Protect, and Manage**

The EA planning discipline that sits between the architecture and the projects — "defining the architectures needed for the business use of information — and the plan to implement them" (Spewak, 1992). -> **Enterprise Architecture Planning (EAP)**

The EAP guiding principle stating that applications manage data and technology supports applications. -> **BUSINESS -> DATA -> APPLICATIONS -> TECHNOLOGY**

The rule of thumb that prevents two projects from changing the same system at the same time. -> **Never run two projects that change the same system at the same time**

The comparison of the As-Is against the To-Be to find what is missing, duplicated, and obsolete; every project must trace back to this. -> **Gap Analysis**

The board that holds checkpoints at initiation, at design, and before go-live, ending each review in approved, approved with conditions, or rejected. -> **Architecture Review Board (ARB)**

A documented exception with an expiry date — otherwise the "temporary fix" becomes permanent technical debt. -> **Deviation exception**

The pitfall described as "beautiful models nobody follows." -> **Ivory-tower architecture**

The four common EA pitfalls. -> **Ivory-tower architecture, no executive sponsor, big-bang scope, stale artifacts**

The four success factors of EA use. -> **Active sponsor, small increments, reviews at project gates, living repository**

The security model where people only get access to what they need for their job (e.g., HR staff can't touch finance data). -> **Role-based access control (RBAC)**

The four main areas of security. -> **Data, Network/Technology, Application, Physical**

The master storage facility for all architectural assets, models, principles, and guidelines; maintains single-source-of-truth visibility. -> **EA Repository**

The repository component that defines the underlying schema, entity types, taxonomy, and relationship rules governing how artifacts connect. -> **Architecture Metamodel**

The repository component that catalogs real-time operational views, target blueprints, and active roadmap transitions. -> **Architecture Landscape**

The EA support tool capability that generates dynamic visual models directly from repository data, keeping diagrams synchronized. -> **Automated Diagramming & Layout**

The EA support tool capability that connects via APIs to cloud platforms, CMDBs, and source repositories to continuously refresh landscape data. -> **Automated Data Ingestion**

The open-source EA tool known for ArchiMate modeling — free, lightweight graphical modeling. -> **Archi (Open Source)**

The EA tool known for comprehensive support for TOGAF, ArchiMate, UML, and SysML. -> **Sparx Enterprise Architect**

The EA tool focused on strategic IT governance & portfolio for large enterprises and financial institutions. -> **Software AG Alfabet**

The EA tool focused on Application Portfolio Management for agile, cloud-native enterprise IT. -> **LeanIX**

The tools used to keep developer compliance: Jira, Confluence, and CI/CD pipelines. -> **SDLC & DevOps alignment tools**

The databases like ServiceNow or AWS Config that reconcile planned architecture with actual runtime reality. -> **CMDB (Configuration Management Database)**

L5 Identification

An approach in software development where we use 'services' to create our applications. -> **SOA (Service-Oriented Architecture)**

A software component we use that is assigned a task; used to make components reusable and modular. -> **Service**

The SOA principle that a service can be used in multiple parts of the system (e.g., Website -> Payment Service <- Mobile App). -> **Reusability**

The SOA principle that changes in one service do not affect other components using the service; services should be stateless. -> **Loose Coupling**

The SOA principle that customers & clients should not be able to see important logic/code. -> **Abstraction**

The SOA principle of dividing a large system into smaller, manageable services, each with its own responsibility. -> **Modularity**

The component that creates, maintains, and provides a service. -> **Service Provider**

The application or system that requests and uses a service. -> **Service Consumer**

A directory of available services, containing information about what services are available and how consumers can access them. -> **Service Registry**

The communication mode where the caller sends a request and waits for an immediate reply before continuing — common with REST APIs. -> **Synchronous communication**

The communication mode where the caller sends a message and keeps working; a response arrives later via a queue or bus. -> **Asynchronous / Message-Based communication**

The runtime discovery mechanism where consumers locate providers through a registry (e.g., UDDI) instead of a hard-coded address. -> **Service Discovery**

An architectural style (not a strict standard) that uses HTTP verbs GET, POST, PUT, DELETE, typically exchanges JSON, and is stateless. -> **REST**

A strict, formal messaging standard where messages are always formatted as XML envelopes with built-in WS-Security. -> **SOAP**

The data format that is verbose, tag-based, common in SOAP messages and legacy enterprise systems. -> **XML**

The lightweight, key-value data format that maps directly to objects and is the default for REST APIs. -> **JSON**

The piece of middleware that sits between services, handling routing, translation, and message delivery. -> **Enterprise Service Bus (ESB)**

The ESB function that converts data formats between systems, such as XML <-> JSON. -> **Transformation**

The ESB function that bridges services using different transport protocols, such as HTTP, JMS, or FTP. -> **Protocol Mediation**

The ESB function that coordinates calls to multiple services in sequence to complete a single business process. ->

Orchestration

The era of software architecture defined by one codebase, one development — simple to start, hard to scale. -> **Monolithic (1990s)**

The era of software architecture where business logic was split into reusable services integrated over the Enterprise Service Bus. -> **SOA (2000s)**

The era of software architecture with small, independently deployable services communicating over lightweight APIs. ->

Microservices (2010s - now)

The modern successor to the ESB — routing, auth, and rate-limiting for cloud APIs. -> **API gateway**

The tools (like Istio) that manage service-to-service communication, adding retries, encryption, and observability. -> **Service mesh**

Docker and Kubernetes as the modern implementation of SOA's "independent service" idea at cloud scale. -> **Containers & orchestration**

L6 Identification

A standard for writing data as plain text, using tags that describe what the data means. -> **XML (eXtensible Markup Language)**

The XML property that means tags tell you what each value is. -> **Self-describing**

The part of an XML document that is always the very first line and declares version and encoding. -> **Prolog**

The part of an XML document that is the outermost element; exactly one per document. -> **Root Element**

XML metadata written inside the opening tag in the form name="value"; the name cannot repeat on the same element. -> **Attribute**

The property of an XML document with exactly one root, no overlapping tags, quoted attribute values, and case-sensitive names. -> **Well-formed (correct syntax)**

Well-formed XML that also follows a schema (only allowed elements, required elements present, values match data types). -> **Valid**

The old schema language written in its own syntax (not XML), with no data types and no namespace support. -> **DTD (Document Type Definition)**

The modern W3C-standard schema language written in XML itself, with real data types (integer, date, decimal, boolean) and namespace support. -> **XSD (XML Schema Definition)**

The XML tool that FINDS/ locates one specific part of the document, written like a folder path. -> **XPath**

The XML tool that SEARCHES XML — what SQL does for tables. -> **XQuery**

The XML tool that CONVERTS an XML file into a different format (HTML, text, or XML). -> **XSLT**

The parser that loads the whole document into memory as a tree; best for small files to modify. -> **DOM**

The parser that streams through and fires events at your code; read once, forward-only, best for very large files. -> **SAX**

The parser where your code asks for the next piece when ready; same efficiency as SAX but you control the loop. -> **StAX**

The approach that converts XML directly into objects (e.g., <student> becomes a student object with getName()). -> **Data Binding (JAXB)**

The gap between the XML tree and a relational database's flat tables. -> **Impedance mismatch**

The kind of XML document with regular, predictable structure where order doesn't matter (invoices, config files). ->

Data-Centric

The kind of XML document with irregular, mixed content where order matters (manuals, contracts, articles). ->

Document-Centric

A normal relational database with XML features added (Oracle XML DB, SQL Server, Db2 pureXML). -> **XML-enabled database**

An XML storage database where the document itself is the unit of storage (eXist-db, BaseX, MarkLogic). -> **Native XML database**

The XML storage technique where the tree is broken into rows and columns — fast joins but the original document no longer exists. -> **Shredding**

The standardized format for global interbank payments. -> **ISO 20022**

The standardized format family for healthcare & clinical data. -> **HL7 / CDA**

The standardized format family for B2B supply chains. -> **RosettaNet / ebXML**

A protocol for exchanging structured information over a network; traditionally uses XML and follows strict rules. -> **SOAP**

An architectural style to design web-based services and APIs; simpler and lighter than SOAP; uses HTTP methods. -> **REST**

The XML-based language that acts like a technical contract for a SOAP service. -> **WSDL (Web Services Description Language)**

The directory service created to publish and discover web services; important early, less used today. -> **UDDI (Universal Description, Discovery, and Integration)**

The family of specifications built on top of SOAP and XML that add security, reliability, and transactions. -> **WS-* technologies**

The WS- *spec that adds authentication, encryption, and message integrity to SOAP messages (often with SAML assertions)*. -> **WS-Security***

The WS- *spec that ensures messages are still delivered correctly even over unstable networks*. -> **WS-ReliableMessaging***

The integration approach where a central process directs how services interact, coordinating from the top down. -> **Orchestration**

The integration approach where services communicate directly peer-to-peer with no single controlling process. -> **Choreography**

L7 Identification

The process of enabling independently designed applications to communicate, exchange data, and work together as a coordinated system. -> **Application Integration**

The direct, custom-built connection between two systems that is generally not reusable and leads to "spaghetti" architecture at scale. -> **Point-to-point integration**

A central hub that manages and routes communication between multiple systems, reducing the total number of connections. -> **Hub-and-spoke integration**

The middleware layer that acts as a common "communication backbone" routing and transforming messages between systems. -> **Enterprise Service Bus (ESB)**

A modern approach where applications expose specific functions through APIs that other systems call. -> **API-based / microservices integration**

Integration where data is synced immediately as changes occur (e.g., stock trading updates balances right after a trade). -> **Real-time integration**

Integration where data is collected and transferred at scheduled intervals (e.g., payroll syncs attendance overnight). -> **Batch integration**

The process of connecting older, often outdated systems with newer systems or technologies. -> **Legacy system integration**

An incremental modernization pattern that minimizes operational risk when migrating legacy systems. -> **Strangler Fig pattern**

A pattern that protects a legacy system from a new system's data model changes. -> **Anti-Corruption Layer (ACL)**

The process of linking separate applications, databases, and hardware platforms across an entire organization to work as a coordinated ecosystem. -> **Enterprise Integration / EAI**

The architecture where systems publish event messages to an enterprise broker and subscribing services react asynchronously. -> **Event-Driven Architecture (EDA)**

The pattern that delivers messages from a single sender to multiple subscribed systems simultaneously. -> **Publish-Subscribe Channel**

The pattern that inspects incoming message contents and routes data to the correct destination based on business rules. -> **Content-Based Router**

The pattern that converts data payloads between incompatible structures (e.g., XML to JSON, EDI to SQL). -> **Message Translator**

The cloud-hosted toolset for connecting applications and data across cloud and on-premises environments. -> **iPaaS (Integration Platform as a Service)**

L8 Identification

The process of linking different IT systems, applications, and data sources so they can work together and share data seamlessly. -> **Enterprise Integration**

The practice of exchanging structured data between systems using XML as a common format. -> **XML Integration**

A standardized way for applications to communicate over a network, regardless of platform or language. -> **Web Services**

An API type that is the most common today, used in web & mobile apps. -> **REST API**

An API type that is enterprise-grade, used in banking/finance. -> **SOAP API**

The API approach that offers flexible querying — request only what you need. -> **GraphQL**

The security standard family that provides message-level security for SOAP. -> **WS-Security**

The formula that distinguishes authentication from authorization. -> **Authentication = "who are you?" vs Authorization = "what can you do?"**

"Well-formed is not the same as" -> **safe**

An enterprise that connects business systems through reusable services. -> **Service-Oriented Enterprise (SOE)**

The SOE component that defines how a service can be accessed. -> **Service Interface**

The SOE component that helps consumers find available services. -> **Service Registry / Broker**

The goal of a Service-Oriented Enterprise. -> **An enterprise that is integrated, flexible, reusable, and easier to manage**

COMPREHENSIVE ACRONYMS LIST

Acronym	Meaning
EA	Enterprise Architecture
ADM	Architecture Development Method
TOGAF	The Open Group Architecture Framework
FEAF	Federal Enterprise Architecture Framework
PRM	Performance Reference Model
DRM	Data Reference Model
SRM	Security Reference Model
BRM	Business Reference Model
ARM	Application Reference Model
TRM	Technical Reference Model
EAP	Enterprise Architecture Planning
PMO	Project Management Office
ARB	Architecture Review Board
RBAC	Role-Based Access Control
EACR	EA Change Request
CONOPS	Concept of Operations
ERD	Entity-Relationship Diagram
IDEF-O	Integration DEFinition / Process modeling (Inputs, Controls, Outputs, Mechanisms)
SWOT	Strengths, Weaknesses, Opportunities, Threats
MDM	Master Data Management
IaaS / PaaS	Infrastructure as a Service / Platform as a Service
BPMN	Business Process Model and Notation
DMZ	Demilitarized Zone
CRM	Customer Relationship Management
ERP	Enterprise Resource Planning
CMDB	Configuration Management Database
ITSM	IT Service Management
SDLC	Software Development Life Cycle
CI/CD	Continuous Integration / Continuous Delivery
SOA	Service-Oriented Architecture
SOE	Service-Oriented Enterprise
ESB	Enterprise Service Bus
WSDL	Web Services Description Language
UDDI	Universal Description, Discovery, and Integration

Acronym	Meaning
REST	Representational State Transfer
SOAP	Simple Object Access Protocol
HTTP / HTTPS	Hypertext Transfer Protocol / HTTP Secure
TLS	Transport Layer Security
XML	eXtensible Markup Language
JSON	JavaScript Object Notation
DTD	Document Type Definition
XSD	XML Schema Definition
XPath / XQuery / XSLT	XML locating / searching / transforming tools
DOM / SAX / StAX	XML parsing models (tree / event-streaming / pull-streaming)
JAXB	Java Architecture for XML Binding (data binding)
CRUD	Create, Read, Update, Delete
EAI	Enterprise Application Integration
iPaaS	Integration Platform as a Service
EDA	Event-Driven Architecture
ACL	Anti-Corruption Layer
ETL	Extract, Transform, Load
OAuth 2.0 / JWT	Open Authorization 2.0 / JSON Web Token
MFA	Multi-Factor Authentication
SLA	Service-Level Agreement
CDN	Content Delivery Network
API	Application Programming Interface
WS-*	Web Services specifications family (security, reliability, transactions)
SAML	Security Assertion Markup Language
B2B	Business-to-Business
ISO 20022	Global interbank payment messaging standard
HL7 / CDA	Healthcare data exchange standard / Clinical Document Architecture
ebXML	Electronic Business using XML
gRPC	Google Remote Procedure Call
AWS EC2 / VPC	Amazon Web Services Elastic Compute Cloud / Virtual Private Cloud
GZIP	File/directory compression tool
XSS	Cross-Site Scripting
DDos	Distributed Denial of Service

TOP THINGS TO MEMORIZE

L2 — Enterprise Architecture Components

1. **4 Phases of EA Implementation** — (I) EA Program Establishment, (II) Framework & Tool Selection, (III) Documentation of the EA, (IV) Use & Maintain the EA.
2. **Zachman Framework** — John Zachman, 1987; 6x6 matrix (36 cells); columns = What/How/Where/Who/When/Why; rows = Planner, Owner, Designer, Builder, Sub-Contractor, Enterprise.
3. **Choosing the right framework** — organize/classify -> Zachman; detailed development process -> TOGAF; government/public sector -> FEAF; flexibility & business alignment -> Gartner.
4. **Four Core EA Components** — Business (foundation), Data/Information, Application, Technology.
5. **Classification of Artifacts** — Catalogs (inventory), Matrices (relationships), Diagrams/Models (visual representation).
6. **Traceability chain** — Business -> Data -> Application -> Technology; used for Impact Analysis.
7. **FEAF Reference Models** — PRM, DRM, SRM, BRM, ARM, TRM.
8. **TOGAF ADM Cycle** — Preliminary, A (Vision) through H (Change Management) + Requirements Management (operates throughout).

L3 — Developing an Enterprise Architecture

9. **Documentation before planning** — plans built without an accurate baseline underestimate cost, timeline, and complexity.
10. **CONOPS Scenario** — good & bad environments from SWOT; Storytelling Technique (Dr. Robert Neilson): Process/People/Technology.
11. **Freeze periods** — freeze current & future views twice a year; changes accumulate 4-5 months; publish in month six.
12. **TOGAF architecture states** — Baseline (as-is), Transition (intermediate), Target (desired future).
13. **EA Management Plan parts (4)** — EA Program Management, Current Architecture Summary, Future Architecture Summary, Glossary & References.
14. **Chief Architect** — manages the EA program, documentation, standards, and configuration management.
15. **Outcome vs Output measures** — outcomes = integration/user satisfaction/IT decisions; outputs = projects completed/scope changes/planning activities.

L4 — Using an Enterprise Architecture 16. **Plan, Protect, and Manage** — Planning & Project Management, Security, Repository & Support Tools. 17. **EAP Guiding Principle** — BUSINESS -> DATA -> APPLICATIONS -> TECHNOLOGY (business-driven, data-driven, never technology-driven). 18. **From Architecture to Projects** — Gap Analysis -> Work Packages -> Business Case -> Roadmap. 19. **Common Pitfalls (4)** — ivory-tower architecture, no executive sponsor, big-bang scope, stale artifacts. 20. **4 Main Areas of Security** — data, network/technology, application, physical. 21. **EA Repository** — central knowledge base; single source of truth for As-Is, To-Be, Transition. 22. **EA Support Tool Capabilities (4)** — automated diagramming, impact & risk analysis, collaborative governance, automated data ingestion.

L5 — All About SOA 23. **SOA Core Principles (4)** — Reusability, Loose Coupling, Abstraction, Modularity. 24. **Main Components of SOA (3)** — Service Provider, Service Consumer, Service Registry. 25. **ESB Core Functions (6)** — Routing, Transformation, Protocol Mediation, Orchestration, Monitoring & Logging, Security Enforcement. 26. **REST vs SOAP** — REST = architectural style, JSON, GET/POST/PUT/DELETE, stateless; SOAP = strict standard, XML envelopes, WS-Security. 27. **Evolution** — Monolithic (1990s) -> SOA (2000s) -> Microservices (2010s-now). 28. **Monolithic vs SOA** — single unit vs independent deployment; one stack vs diverse; whole-app scaling vs targeted scaling; full-system failure vs isolated failure. 29. **SOA vs Microservices** — ESB-mediated vs direct REST/gRPC; shared data vs per-service data; centralized vs decentralized governance. 30. **SOA's Legacy** — API gateways, service mesh (Istio), containers & orchestration (Docker/Kubernetes), hybrid & legacy integration.

L6 — Integrating Technology 31. **XML properties** — self-describing, extensible, platform neutral, data not design. 32. **XML document parts** — prolog, root element, child element, attribute, element content, nesting. 33. **Well-formed vs Valid** — well-formed = correct syntax; valid = well-formed + follows a DTD/XSD. 34. **DTD vs XSD** — DTD = old, own syntax, no data types; XSD = W3C standard, written in XML, real data types, namespaces. 35. **XPath finds, XQuery searches, XSLT converts**. 36. **Reading XML** — DOM (tree, edit), SAX (streaming, read-only), StAX (pull streaming), JAXB (data binding). 37. **Storing XML** — XML-enabled DB, Native XML DB, Hybrid/XML column; shredding / CLOB / native tree. 38. **XML integration standards** — ISO 20022 (interbank payments), HL7/CDA (healthcare), RosettaNet/ebXML (B2B supply chains). 39. **Web service tech** — SOAP, REST, XML, JSON, HTTP/HTTPS, WSDL (the contract), UDDI (the directory).

L7 — Integrating Applications 40. **Integration styles** — Point-to-Point, Hub-and-Spoke, ESB, API-based/Microservices. 41. **Real-time vs Batch** — real-time = immediate sync (stock trading); batch = scheduled intervals (overnight payroll). 42. **Legacy integration approaches (5)** — API Wrapping, Middleware Connectivity, ESB, iPaaS, ETL & Data Synchronization. 43. **Legacy challenges (6)** — architectural mismatch, data inconsistency, security gaps, skill shortages, missing documentation, downtime risks. 44. **Enterprise integration architectures** — ESB, Hub-and-Spoke, SOA, Event-Driven (EDA).

L8 — Enterprise Integration 45. **Enterprise Integration** — connecting, not replacing, existing systems. 46. **XML integration best practices** — validate everything (well-formed is not safe), use HTTPS, least privilege, block XML attacks. 47. **SOAP vs REST (L8)** — SOAP = protocol/XML-only/built-in security/slower/enterprise; REST = style/JSON/implementation-dependent/faster/web & mobile. 48. **Auth vs Authorization** — Authentication = "who are you?"; Authorization = "what can you do?" 49. **SOE Main Components (5)** — Provider, Consumer, Registry/Broker, Interface, ESB. 50. **WS-* specs** — WSDL (describe), UDDI (discover), WS-Security (secure), WS-ReliableMessaging (reliable).

QUICK CRAM SHEET (HIGHEST-YIELD FACTS)

EA Methodology vs Framework — Methodology = HOW the EA program is run; Framework = WHAT structure organizes the EA.

Zachman — 1987, John Zachman, 6x6 = 36 cells, classification schema (not methodology), columns = What/How/Where/Who/When/Why, rows = Planner/Owner/Designer/Builder/Sub-Contractor/Enterprise.

FEAF models mnemonic — Performance, **D**ata, **S**ecurity, **B**usiness, **A**pplication, **T**echnical.

4 EA layers — Business -> Data -> Application -> Technology (Business is the foundation).

Artifacts — Catalogs = inventory, Matrices = relationships, Diagrams/Models = visual/structure.

ADM cycle order — Preliminary -> A Vision -> B Business -> C Info Systems -> D Technology -> E Opportunities -> F Migration -> G Governance -> H Change (+ Requirements Mgmt throughout).

2 KEY EA numbers — Zachman 36 cells; Freeze periods twice a year (publish month 6).

EAP principle — Business -> Data -> Applications -> Technology; business-driven and data-driven, NEVER technology-driven.

4 EAs to projects — Gap Analysis, Work Packages, Business Case, Roadmap.

Plan -> Protect -> Manage (the three EA usage areas).

SOA 4 principles — Reusability, Loose Coupling, Abstraction, Modularity.

SOA 3 components — Provider, Consumer, Registry.

ESB 6 functions — Routing, Transformation, Protocol Mediation, Orchestration, Monitoring & Logging, Security Enforcement.

REST vs SOAP — REST: style, JSON, fast, web/mobile; SOAP: protocol, XML, secure, banking/enterprise.

Architecture eras — Monolithic (90s) -> SOA (2000s) -> Microservices (2010s-now).

Microservices fixes over SOA — no central bus (ESB = single point of failure), per-service data, autonomous teams, isolated failure.

SOA legacy — API gateways, service mesh (Istio), Docker/Kubernetes, hybrid integration.

XML 4 properties — self-describing, extensible, platform neutral, data not design.

XML parts — prolog, root, child, attribute, content, nesting.

Well-formed vs valid — well-formed = syntax correct; valid = well-formed + follows schema.

DTD vs XSD — DTD old/no types; XSD modern/W3C/real types/namespaces.

XML 3 tools — XPath finds, XQuery searches, XSLT converts.

XML parsing — DOM (tree), SAX (events), StAX (pull), JAXB (objects).

Impedance mismatch — XML tree vs database tables.

3 ways to store XML — XML-enabled DB, native XML DB, hybrid/XML column.

Integration styles — Point-to-Point, Hub-and-Spoke, ESB, API-based, Event-driven.

Legacy vs New — legacy = cheaper, less disruption, less training, limited scalability; new = pricier, more downtime, more training, highly scalable.

EAI objectives — data consistency, system interoperability, business process automation, real-time visibility.

Integration patterns — messaging channels, content-based router, message translator, publish-subscribe.

Web service stack — SOAP + WSDL + UDDI (+ WS-* for security/reliability).

Authentication vs Authorization — "who are you?" vs "what can you do?"

SOE components (5) — Provider, Consumer, Registry/Broker, Interface, ESB.

XML integration mindset — structure clearly, validate & secure every hop, optimize performance, handle errors with clear feedback, design for the reader, trust nothing by default.

End of Reviewer — generated from SAI Reference (L2-L8). Study hard — you've got this!